TOS Network

The Open System for the Agentic Internet

Foundational Infrastructure for the Agentic Internet

Mission: Connect Every AI Agent

TOS Network

2026 Agentic Internet Edition

Abstract

AI agents are moving from assistants that recommend actions to autonomous participants that discover counterparties, negotiate terms, invoke services, spend bounded budgets and coordinate across organizational boundaries. Existing APIs, agent interfaces and payment rails are necessary, but they do not by themselves establish who an agent represents, what it may do, what both parties accepted, how execution is bounded, what evidence must be returned or how failure is resolved.

TOS Network is being built as the **foundational infrastructure**—**The Open System**—**for the Agentic Internet**. Its mission is **Connect Every AI Agent**. The common lifecycle is **Identity** → **Discovery** → **Intent** → **Negotiation** → **Agreement** → **Bounded Execution** → **Evidence/Receipt** → **Settlement**. An Intent expresses a revocable objective; it does not by itself create an obligation. An Agreement records the exact terms accepted by the relevant parties. Only then is work delegated to a bounded execution path.

Agentic Service Operations are execution-layer primitives within this larger infrastructure, not the definition of TOS as a whole. Each versioned operation binds an input and output contract to capability scope, replay domain, resource envelope, maximum budget, metering, receipt semantics and settlement policy. Communication, model and tool calls, storage, events, software work, booking, commerce and physical-edge work can use the same accountable envelope while retaining domain-specific execution and safety rules.

TOS reuses HTTP, MCP, A2A, OpenAPI and compatible discovery systems rather than replacing them. Models may interpret user goals, but deterministic policy and explicit authority decide what can be accepted and executed. Providers retain control over admission, hardware, models, data, pricing and local safety. Private payloads and workloads remain off-chain; TOS Core is used for shared identity, commitments, disputes and final settlement where independent parties need a tamper-resistant source of truth.

The implemented foundation, TOS Core, is an actor-model blockchain with native TVM execution, value-bearing asynchronous messages, masterchain-rooted consensus, dynamic sharding and ADNL/DHT/RLDP networking. The open Intent, Agreement and Agentic Operation protocols, SDKs, conformance suites and most service profiles are partial or planned. Artificial Intelligence Proof of Work (AIPoW) is feature-gated off by default: token incentives must not substitute for product-market fit, organic paid demand or sustainable unit economics.

Document Status

This paper describes both released foundations and architectural direction. “Implemented” means released code contains the relevant node, contract, tooling or test foundation. “Partial” means

useful primitives or one profile-specific implementation exist, but the generic interoperable product surface is incomplete. "Planned" identifies work that still requires specification, implementation, conformance testing and independent review.

Contents

1 Executive Summary

The Internet connected computers. TOS is being built to connect autonomous agents.

TOS Network is the proposed foundational infrastructure for the Agentic Internet: an open system through which agents operated by different people, organizations and runtimes can find one another, communicate, form precise commitments, act within delegated limits, exchange evidence and settle outcomes. Its mission is:

Connect Every AI Agent.

An API may describe how to format and transport a request, yet it usually does not provide a portable answer to the questions autonomous counterparties must resolve before acting:

- who is making the request and under whose authority;
- how a goal becomes a proposal and then a mutually accepted Agreement;
- which exact operation and revision will execute that Agreement;
- what resources the operation may consume;
- how much the caller is willing and permitted to spend;
- how replay, expiry, cancellation and retries are bounded;
- what evidence and receipt the provider must return; and
- how payment, refund, dispute and final settlement occur.

TOS Network is being built to standardize this missing infrastructure. The primary lifecycle is not a payment call or an economic opcode in isolation:

```
Identity -> Discovery -> Intent -> Negotiation -> Agreement
  -> Bounded Execution -> Evidence/Receipt -> Settlement
```

Identity establishes the accountable principal and its delegated authority. Discovery returns candidates, not permission. Intent expresses a revocable objective. Negotiation refines price, scope, timing, evidence and recourse. Agreement is the exact accepted commitment. Agentic Service Operations then provide typed, metered execution primitives. Evidence and Receipts support verification, reputation, dispute and final settlement.

Every action is typed. Every authority is bounded.
Every resource is metered. Every result is receipted.
Every cost is accountable.

The architecture has four interoperable infrastructure layers surrounded by the participants and economies that use them:

1. **Agent Access, Coordination and Discovery.** Identity-aware entry points, authentication, authorization, capability publication and search, matching, quoting, spending policy, Agreement formation, job coordination, events and result delivery.
2. **Autonomous Runtime and Provider Execution.** Agent runtimes, provider and worker runtimes, protocol and tool adapters, isolated execution, resource accounting, evidence capture and result generation.
3. **Trust, Evidence and Economy.** Identity and capability commitments, reputation references, escrow, payment, Receipt verification, proof, dispute and optional demand-aligned contribution incentives.
4. **Decentralized Network Foundation.** Validators, peer-to-peer propagation, consensus, the immutable ledger, smart contracts, event logs, state synchronization and finality.

Users, personal and enterprise agents, autonomous applications and machines enter through the access layer. Model, API, compute, storage, human and physical-service providers connect to the runtime layer. Node operators, stakers and governance participants maintain the network foundation. Applications and institutions may compose all four layers without becoming a new source of protocol authority.

Only TOS Core and several reusable actors are substantially implemented as common foundations. Profile-specific messaging and mailbox primitives exist, while the generic operation registry, common request and receipt envelope, provider conformance suite and several economic profiles remain partial or planned. This distinction is deliberate. Validators do not execute application workloads, and adding a model, messenger feature, storage backend or commercial profile must not require a consensus upgrade.

TOS does not seek to replace HTTP, MCP, A2A, OpenAPI, ARD or application-specific protocols. Those systems can carry, describe or discover an interface. TOS adds portable identity, authority and commitment around the action: who may propose, what was accepted, who may call, within what limits, at what maximum cost, with what evidence and through what settlement path.

1.1 Strategic Positioning

The stable top-level positioning is:

> **TOS Network is the foundational infrastructure—The Open System—for the Agentic Internet.**

The protocol-level expression is that TOS enables autonomous agents to discover, communicate, coordinate, transact, execute and settle across organizational and platform boundaries. The application-level expression is that an ordinary Internet action can become typed, authorized, metered, receipted and settleable.

A useful architectural shorthand is that TOS provides the **economic syscall layer for the Agentic Internet**. This does not mean a TOS validator executes every external action as a kernel instruction. It means an agent can treat an independently provided service as a well-defined operation with portable semantics, bounded authority, resource accounting and a verifiable outcome.

HTTP standardized transport. DNS standardized naming. OpenAPI, MCP and A2A describe different forms of interface and interaction. ARD makes resources discoverable. TOS aims to standardize the economically accountable action that follows discovery.

The unit of coordination is therefore an accepted, policy-compliant Agreement that may be fulfilled by one or more Agentic Service Operations. It is not an hour of unidentified hardware, an unbounded API call or an unauthenticated message. A consumer and provider agree on an exact operation revision, input and output contract, deadline, region, data-egress policy, resource limit, maximum charge, evidence level and recourse. The provider or target agent retains the right to apply local admission and safety policy, executes within the accepted bounds, returns a signed receipt and settles according to the Agreement.

1.2 Why Now

Four curves are converging.

1. **Agents can act.** Tool-using models and durable agent runtimes can now plan, call APIs and complete multi-step workflows with decreasing human supervision.

2. **Interfaces are becoming machine-readable.** OpenAPI, MCP, A2A and emerging discovery formats reduce integration cost, but they do not provide one portable economic and authority contract.
3. **Machine-native payment is becoming practical.** Programmable wallets, stablecoin settlement and payment-bearing HTTP patterns demonstrate that software can buy a service without a human checkout flow. Payment proves transfer; it does not by itself prove delegated authority, bounded consumption, correct execution or recourse.
4. **Verification and abuse become the scarce resources.** As the marginal cost of generating requests and outputs falls, spam, unauthorized spending, unverifiable work and autonomous retry loops become more expensive for recipients and operators.

Public a16z crypto analysis describes the same market transition in adjacent terms: agents are becoming economic actors, while portable identity, permissions, payment and trust remain fragmented. [1] The TOS thesis is that the missing product is not another model or payment button. It is the enforceable contract around an autonomous action.

1.3 Beachhead, Expansion and Network Flywheel

The architecture is horizontal, but the go-to-market sequence must be narrow. The first external acceptance target is **machine-checkable software work**: independent agents form an Agreement over an exact repository state, bounded task, tests, deliverables, evidence and recourse, then execute and verify the outcome without sharing one runtime. This profile exercises the full Intent-to-Settlement lifecycle and produces evidence that independent implementations can inspect.

The first commercial service wedge is **paid model and tool invocation**: an agent discovers an independent provider, receives a live quote, delegates a bounded budget, invokes a typed operation, verifies a signed usage receipt and settles without creating a bilateral account integration. This wedge has an immediate buyer, a measurable unit of work and a clear comparison against API keys, monthly invoices and payment-only alternatives.

Messenger and Mailbox form the second reference family. They are strategically useful because they stress-test identity, recipient policy, delegation, retention, delivery evidence, refundable attention bonds and machine-scale spam resistance. They should validate the common operation model after the compute wedge rather than divide initial go-to-market focus.

The reference applications built by TOS must be the protocol's first demanding customers. They should use the same public envelopes, SDKs and receipts offered to third parties; an internal privileged path would hide integration cost and weaken the evidence for adoption. External design partners should include at least one agent builder and multiple unrelated service providers whose requirements can shape the first frozen profile.

If these beachheads work across independent implementations, the intended flywheel is:

```
open operation definitions -> competing providers -> portable receipts
  ⟶ better matching, trust, price and availability -> more demand
      ⟶ more providers and operation profiles.
```

This follows a broad–narrow–broad market-design sequence: define the horizontal primitive, win a high-value transaction category, then expand through adjacent profiles. Open protocols compound

[1] Christian Crowley et al., "The missing infrastructure for AI agents: 5 ways blockchains can help," a16z crypto, April 16, 2026: https://a16zcrypto.com/posts/article/5-ways-blockchains-help-ai-agents. This reference provides market context only and does not imply endorsement of TOS Network.

through third-party embedding, so integrations and conformance matter more than a closed catalog. [2]

1.4 Defensibility and Value Capture

The defensible asset is not token issuance, a proprietary model or a closed list of services. It is the interoperable graph of operation identities, exact revisions, provider descriptors, capabilities, quotes, receipts, disputes, settlement history and measured reliability. Every additional conforming provider increases choice; every additional verified receipt improves admission and matching; every additional integration makes the operation vocabulary more useful to agents elsewhere.

Openness is therefore a distribution strategy, not the absence of a moat. Durable advantage must come from becoming embedded in agent runtimes and provider SDKs, producing the safest and lowest-friction completion path, and accumulating credible cross-provider performance data. If providers can multi-home with no loss of history, trust or liquidity, the network effect will be weak. If agents can carry identity, policy and receipts while providers compete on the same definitions, TOS can become neutral coordination infrastructure rather than one more marketplace application.

TOS Core captures protocol demand through transaction and message fees, persistent actor state, escrow and dispute execution, registry and role bonds, receipt or channel commitments, and validator security. Service-provider revenue remains distinct. The network should permit applications to quote familiar units or use sponsored and batched payment policies where appropriate; TOS must earn default settlement through predictable cost, finality, developer experience and credible neutrality rather than by making every payload or price TOS-only.

1.5 Why a Dedicated TOS Settlement Layer

A new blockchain is not justified merely because agents can transact. General-purpose chains, stablecoins and payment-bearing HTTP can settle many calls. TOS Core is justified only if its actor-native asynchronous execution, value-bearing messages, persistent agent policy, parallel Service Actors and sharded settlement make the full authorization-to-receipt flow safer, cheaper or easier to operate across independent parties.

That claim is falsifiable. Reference profiles must publish end-to-end latency, failure and recovery behavior, settlement cost as a share of operation value, channel or batch efficiency, state growth, validator decentralization and equivalent deployment comparisons. If a general-purpose chain plus the open TOS envelopes performs as well, the operation protocol may still be useful, but the investment case for TOS Core and the native asset is materially weaker.

1.6 Falsifiable Adoption Gates

Before describing this system as product-market fit or a network effect, public reporting should show at least:

- weekly and monthly cohorts of unrelated active payers, providers and agent builders;
- quote-to-paid-completion conversion, failed-match rate and time to successful match;
- payer and provider retention without token rewards or protocol-funded demand;
- organic settled value and its share of all settled value;
- service revenue, protocol fees and subsidies as separately reported economic flows;
- operation completion, timeout, cancellation and dispute-overturn rates;

[2] Scott Duke Kominers, "3 takeaways on market design for web3 builders," a16z crypto, September 19, 2024: https://a16zcrypto.com/posts/article/3-takeaways-on-market-design-for-web3-builders.

- resource-estimate error between quoted limits and actual metered use;
- receipt verification and cross-provider conformance rates;
- the share of operations and providers reached through third-party integrations;
- spam rejection, bond refund and false-positive rates by communication profile;
- concentration of settled value and AIPoW score after common-control aggregation; and
- issuance divided by organic settled value when AIPoW is active.

Subsidized circular volume, address count, message count and declared capacity are not substitutes for these measures. Section 16.7 defines the demand and reward quantities used by AIPoW reporting.

1.7 Scale of the Opportunity

If autonomous software continues to multiply, machine-to-machine operations become an economic layer distinct from human-facing digital commerce. The addressable activity is not limited to model inference. It includes communication, storage, event delivery, tools, booking, procurement, verification, human work and physical services. TOS Network's design goal is to provide the identity, authorization, metering, receipt and settlement infrastructure this layer requires.

The venture-scale outcome is therefore not "another AI marketplace." It is an open transaction fabric used by many agent products and service networks, with TOS Core serving as a preferred neutral settlement domain. The opposite outcome is also explicit: if agents stay inside a few vertically integrated platforms, if payment-only protocols absorb the authority and receipt layer, or if buyers do not value portable recourse, the addressable independent network will be smaller. This is a structural thesis, not a claim about currently realized transaction volume, protocol revenue or token value.

2 From Intent to Settlement

TOS treats an autonomous workflow as a sequence of distinct commitments. Collapsing these states into one "agent request" makes it difficult to determine when authority began, what was actually accepted, which evidence is relevant or whether payment is final.

```
Identity -> Discovery -> Intent -> Negotiation -> Agreement
  -> Bounded Execution -> Evidence/Receipt -> Settlement
```

2.1 Identity and Discovery

Identity identifies an agent, owner, provider, verifier or other principal and binds the keys, roles and delegation policy relevant to the workflow. An agent identity must not imply unrestricted owner authority. Key rotation, revocation, scope, expiry and delegation depth are part of the authority model.

Discovery returns possible counterparties, capabilities, endpoints and operation profiles. It is informative rather than authoritative: appearing in a catalog, index, DHT record or search result does not prove current capacity, price, permission, trustworthiness or safety. Clients re-verify signed descriptors, capabilities, quotes and settlement destinations before committing funds or work.

2.2 Intent, Negotiation and Agreement

An **Intent** is a revocable expression of a desired outcome and constraints. It may name a task, deadline, budget ceiling, privacy policy, preferred evidence or acceptable providers, but it is not

itself an authorization to spend or execute and does not create an obligation for another party.

Negotiation converts an Intent into concrete proposals. Parties or their authorized agents can refine scope, price, inputs, outputs, resource ceilings, deadlines, cancellation, evidence, dispute forum and settlement method. A model may interpret natural-language goals and propose terms; deterministic policy must decide whether those terms fit the principal's authority and budget.

An **Agreement** is the exact, authenticated set of terms accepted by the relevant parties. It binds identities, roles, selected operation revisions, capability references, payload or schema commitments, budgets, evidence policy, recourse and expiry. An Agreement may authorize one operation, a sequence, a conditional workflow or a reusable allowance. No implementation should infer acceptance merely from discovery, conversation, possession of an endpoint or an unsigned model output.

2.3 Bounded Execution, Evidence and Settlement

Agreement does not require validators to execute the real-world workload. Providers perform model inference, communication, storage, software work, database access and physical action off-chain under local admission, security and safety policy. Agentic Service Operations are the typed execution primitives used to carry out the accepted terms. Every externality—cost, compute, storage, delivery attempts, attention, retries or physical reach—is bounded by capability, quota, budget, bond or another enforceable policy.

Evidence is evaluated only under the policy named by the Agreement. A Receipt records a signed statement about status, metered use, result commitments, evidence references and settlement state; it proves no more than its schema, signer and evidence policy authorize it to prove. A stored receipt is not a delivery receipt, an execution receipt is not necessarily a correctness proof, and a payment receipt is not proof that the underlying result was valid.

Settlement applies the accepted payment, refund, bond, dispute and finality rules. It may use direct TOS transfers, escrow, prepaid balances, channels, batches, sponsorship or a zero-price allowance. The invariant is accountable completion, not a mandatory per-call fee.

2.4 Why an API Call Is Not an Agent Contract

A conventional API call primarily specifies endpoint syntax and transport behavior. The caller may still depend on provider-specific accounts, OAuth scopes, billing, rate limits, error meanings, retry policy and evidence. Two APIs that both "send a message" or "invoke a model" can be economically and operationally incompatible.

An Agentic Service Operation adds the information required for autonomous action across organizational boundaries. It binds the requested behavior to identity, authority, resource limits, economic policy and a verifiable result.

```
API call + capability + budget + metering + receipt + settlement
                              =
                Agentic Service Operation
```

The operation abstraction is intentionally transport-neutral. It may be carried through RLDP/TOS Sites, HTTPS, QUIC, WebSocket, a message queue, a local IPC channel or another agreed binding. Transport success does not imply operation authorization or settlement success.

2.5 Agentic Service Opcode

An **Agentic Service Opcode** is the architectural name for a versioned service-layer action such as:

`messenger.send@1` `mailbox.deposit@1` `model.infer@1` `storage.put@1`

The normative protocol field is `operation_id`. "Opcode" is useful because the operation behaves like a syscall from the agent's perspective: it has defined input, authority requirements, cost and return semantics. It is not a claim that the action is a native TVM instruction or that validators perform the external workload.

Multiple independent providers may implement the same operation revision. An agent can discover candidates, compare quotes and evidence policy, authorize one within a bounded budget, receive a standardized receipt and switch providers without learning a completely different economic protocol.

2.6 Opcode Registry and Maturity

This edition names **25 distinct Agentic Service Opcode families**. These identifiers are an architectural vocabulary and a development registry; naming an opcode in this paper does not mean that its revision has been frozen as an interoperable TOS standard. Earlier drafts also used `model.infer@2` as an illustrative revision. This edition normalizes the illustrative model-inference entry to `model.infer@1`; a future incompatible semantic change would advance the immutable revision in the normal way.

Opcode maturity is tracked independently from the implementation status of the underlying TOS Core contracts and networking primitives:

Maturity	Meaning
Design	The operation name and intended semantics are described, but the canonical wire contract and conformance vectors are not complete.
Partial	One or more profile-specific implementations or reusable primitives exist, but the complete canonical operation contract is not frozen.
Frozen	Canonical identifier, revision, schemas, authority rules, metering, errors, Receipt semantics and success/failure vectors are published and immutable for that revision.
Implemented	A reference implementation passes the frozen conformance suite for that exact revision.
Interoperable	At least two independently authored implementations exchange and verify equivalent requests, results and Receipts for the frozen revision.
Production	The interoperable revision has additionally passed the applicable security, release, operational and public-network gates.

As of this edition, **zero Agentic Service Opcode revisions have reached the Frozen, Interoperable or Production maturity levels defined above**. Messenger and Mailbox have the strongest profile-specific implementation foundations, but their generic operation contracts remain Partial. The remaining opcode families are Design/Planned unless a released profile specification says otherwise. This prevents an implemented low-level behavior from being mistaken for a frozen Internet-wide semantic contract.

Opcode	Maturity	Current scope
`messenger.send@1`	Partial	Messaging-plane foundations exist; generic commercial and cross-provider contract not frozen.
`messenger.introduce@1`	Design	Introduction/attention-bond semantics remain profile specification work.
`messenger.acknowledge@1`	Partial	Acknowledgement foundations exist; portable Receipt semantics are not frozen.
`mailbox.deposit@1`	Partial	Capability-bound deposit and stored-acknowledgement foundations exist.
`mailbox.read@1`	Partial	Capability-bound read foundations exist; generic provider conformance is incomplete.
`mailbox.delete@1`	Partial	Capability-bound delete foundations exist; generic provider conformance is incomplete.
`mail.send@1`	Design	Mail-specific domain and anti-abuse profile remains planned.
`model.infer@1`	Design	Model-provider quote, metering, evidence and Receipt profile remains planned.
`embedding.create@1`	Design	Model/embedding profile remains planned.
`speech.transcribe@1`	Design	Speech profile remains planned.
`tool.invoke@1`	Design	Generic tool invocation profile remains planned.
`workflow.execute@1`	Design	Workflow composition profile remains planned.
`storage.put@1`	Design	Storage lease and durability profile remains planned.
`storage.get@1`	Design	Storage retrieval profile remains planned.
`storage.renew@1`	Design	Storage renewal profile remains planned.
`storage.delete@1`	Design	Storage deletion/expiry semantics remain planned.
`event.publish@1`	Design	Event fan-out, backlog and settlement profile remains planned.
`event.subscribe@1`	Design	Subscription, replay and backlog profile remains planned.
`commerce.quote@1`	Design	Commerce domain state machine remains planned.
`commerce.order@1`	Design	Commerce order/fulfillment profile remains planned.
`booking.reserve@1`	Design	Reservation, deposit and cancellation profile remains planned.
`booking.cancel@1`	Design	Booking cancellation/refund semantics remain planned.
`delivery.request@1`	Design	Delivery and fulfillment profile remains planned.

Opcode	Maturity	Current scope
`human.task@1`	Design	Human-service evidence and dispute profile remains planned.
`task.post@1`	Design	Task-market operation wrapper remains planned above the implemented Task Escrow actor.

The registry is intentionally extensible. Third parties may define new operation namespaces, but a TOS publication must state its maturity explicitly. A Design or Partial identifier must not be advertised as a frozen TOS standard, and a Frozen identifier must not be advertised as Interoperable until an independent implementation demonstrates the same semantics.

2.7 Three Opcode Layers

This paper distinguishes three different meanings of opcode.

Layer	Examples	Execution boundary	Pricing boundary
TVM instruction opcode	`SENDMSG`, `SETCODE`	Deterministic validator execution	Gas, cells, message forwarding and persistent storage
Contract message opcode	`AGENT_TASK_SEND` and contract ABI operations	Receiving actor interprets the message body	Contract execution, attached value and message fees
Agentic Service Opcode	`messenger.send@1`, `model.infer@1`	External service, relay, terminal or another agent	Profile-specific usage, quote, allowance, bond and settlement

The layers may cooperate in one workflow. An off-chain `mailbox.deposit@1` operation can use an Agent Account for authorization, a Service Actor or escrow for payment, and ordinary TVM instructions for the on-chain state transition. They remain different abstractions and must not share ambiguous documentation or numeric registries.

2.8 Open Identity, Namespace and Versioning

A global centrally assigned 32-bit number is not sufficient for an open Internet-scale operation namespace. The full operation identity should be globally unambiguous, versioned and bound into signatures. Examples include:

```
urn:tos:op:messenger.send:1
urn:tos:op:mailbox.deposit:1
urn:tos:op:example.com:weather.forecast:3
```

The exact syntax is specification work, but the following rules are required:

- the full operation identifier and revision are authenticated;
- incompatible semantics require a new immutable revision;
- schemas, metering dimensions, receipts and error meanings are content-addressed;
- domain-anchored or otherwise verifiable namespaces permit open extension;

- no single registry is mandatory for all operation publication; and
- compact numeric wire codes may be negotiated only inside an authenticated session after the full identifier has been agreed.

An operation definition is not a provider advertisement. The operation says what behavior and semantics mean. A provider descriptor says that a particular identity implements that operation under a particular endpoint, quote policy and evidence level.

2.9 Formal Operation Definition

A versioned operation definition can be represented as

$$\mathcal{O} = (id, v, S_{\text{in}}, S_{\text{out}}, K, M, R, E),$$

where:

- id is the operation identifier and v its immutable revision;
- S_{in} and S_{out} are canonical input and output schemas;
- K defines capability, delegation and target-policy rules;
- M defines resource dimensions and metering;
- R defines receipt, evidence and settlement semantics; and
- E defines errors, cancellation, retries and terminal states.

A request can be modeled as

$$Q = (op, caller, target, capability, nonce, expiry, \mathbf{u}_{\max}, C_{\max}, h(payload), correlation),$$

where $\mathbf{u}_{\max}$ is a vector of resource ceilings and $C_{\max}$ is the maximum authorized charge. The payload may remain encrypted and off-chain; the authenticated envelope binds its digest.

A provider quote can be represented as

$$\mathcal{Q} = (provider, op, v, \mathbf{p}, \mathbf{u}_{\max}, C_{\max}, t_{\text{expiry}}, policy),$$

where $\mathbf{p}$ prices the operation's resource dimensions. A receipt can be represented as

$$R = (status, \mathbf{u}_{\text{actual}}, C_{\text{actual}}, h(result), evidence, settlement, \sigma_{\text{provider}}).$$

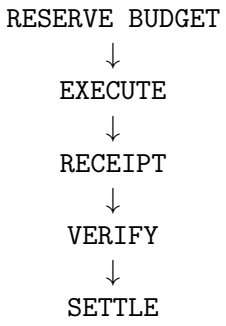
For accepted operations,

$$C_{\text{actual}} \leq C_{\max}.$$

A profile may also define minimum charges, reservation charges, cancellation fees, refunds, sponsorship or a refundable bond. The accepted quote and operation definition determine the meaning; a receipt cannot invent new dimensions after execution.

2.10 Operation Execution Lifecycle

A common lifecycle allows different profiles to share tooling and reasoning:



This is the execution sub-lifecycle of an accepted Agreement, not a substitute for Intent and negotiation. Every step is asynchronous. A quote may expire. Authorization may fail after discovery. Admission may reject an otherwise affordable call. Execution may return a partial result. Receipt verification may trigger dispute instead of settlement. These are explicit states, not hidden exceptions.

Long-running operations require stable operation identity, idempotency keys and restart recovery. Retrying a request must not silently duplicate a payment, message, reservation, physical action or task.

2.11 Metering and Charging Are Distinct

Every meaningful operation must have bounded and attributable resource use, but not every operation must charge a non-zero amount.

A valid policy may use:

- direct payment in TOS;
- prepaid credit or subscription allowance;
- organization, recipient or application sponsorship;
- a trusted-party free quota;
- congestion-dependent pricing;
- a refundable anti-spam or performance bond;
- batched settlement or a payment channel; or
- deferred aggregate settlement.

The invariant is not "every action pays." The invariant is:

> **No caller may impose unbounded external cost without identity, authorization, quota, budget or bond.**

Even a zero-price operation remains metered. Its provider can account for bytes, time, delivery attempts, compute, storage or recipient attention; enforce limits; produce a receipt; and attribute abuse to an identity and capability.

2.12 Representative Operation Profiles

Operation	Principal metering dimensions	Typical control and settlement policy
`messenger.send@1`	Ciphertext bytes, recipients, delivery attempts, priority	Recipient capability, contact quota, introduction bond, relay quote

Operation	Principal metering dimensions	Typical control and settlement policy
`mailbox.deposit@1`	Bytes, retention duration, replica count	Prepaid storage allowance, capacity ceiling, signed stored receipt
`mailbox.read@1`	Requests and outbound bytes	Read capability, device binding, rate limit
`mail.send@1`	Body and attachment bytes, delivery class	Sender reputation, domain policy, refundable attention bond
`model.infer@1`	Input and output tokens, accelerator time, latency class	Maximum budget, provider quote, usage and evidence receipt
`tool.invoke@1`	Calls, duration, external provider cost	Capability, deadline, escrow, normalized result and errors
`storage.put@1`	Bytes, duration, redundancy and egress	Lease, prepayment, deletion and durability policy
`event.publish@1`	Events, subscribers, fan-out and retention	Publisher quota, congestion price and bounded replay
`booking.reserve@1`	Resource quantity and lock duration	Refundable deposit, cancellation window and confirmation receipt
`task.post@1`	Budget, deadline and verification tier	Escrow, claimant bond, result receipt and dispute policy

These identifiers are illustrative until frozen by their respective specifications. Their purpose is to show that communication, compute, storage and commerce can share one economic operation model without sharing one workload implementation. The authoritative maturity summary for the identifiers named in this edition is Section 2.6.

3 Design Principles

3.1 Typed Operations, Not Bare Endpoints

Agents request a versioned operation and declared outcome. A URL, host, model name, device class or raw hardware claim may help discovery and scheduling, but it is not the operation contract. Providers expose bounded service behavior rather than an unrestricted shell, container socket, accelerator or actuator.

3.2 Actor First

Every account is an actor. Agent Accounts, Task Escrows, capability entries, service actors and verifier actors are ordinary native accounts with isolated state. There is no privileged global application process.

3.3 Asynchronous by Default

Cross-actor and cross-provider workflows use messages, callbacks, deadlines, receipts and retries. A sender cannot assume synchronous execution or multi-step atomicity. Contracts and service profiles expose explicit intermediate states and deterministic failure behavior.

3.4 Authority Must Be Explicit

Owner, controller, creator, assigned agent, recipient, verifier, service, relay and fee-payer roles are distinct. Every state transition, resource reservation and transfer identifies its authority. Discovery, possession of an endpoint and payment capacity do not grant execution authority.

3.5 Models Interpret; Deterministic Policy Authorizes

A model may translate natural language into an Intent, propose an execution plan or evaluate non-deterministic evidence. It must not silently expand capability, budget, target, delegation depth or physical reach. The accepted Agreement and deterministic policy checks remain the authority boundary. Ambiguity, unavailable evidence or policy conflict produces refusal, renegotiation or human escalation rather than inferred permission.

3.6 Meter the Externality

The initiator should fund or otherwise hold authorization for the resource and attention cost it imposes. Bytes, storage duration, delivery attempts, compute, tool expense, reservation time and retries are profile-specific resources. A free allowance is still finite and attributable.

3.7 Verification Without Payload Bloat

The chain is authoritative for funds, permissions, deadlines and settlement where shared trust is required. Large messages, model prompts, outputs, datasets, sensor streams and private credentials remain off-chain. Contracts store hashes, signatures, attestations and evidence references that bind external work to accepted operations.

3.8 Local Control, Privacy and Safety

Providers retain custody of hardware, data, models and operational policy. Encrypted communication payloads and private execution context remain outside public consensus. A physical terminal continues approved local work while disconnected and keeps blockchain consensus outside hard real-time and actuator-control loops. Participation in TOS does not transfer ownership or control of a provider's infrastructure; the provider may reject work that violates current admission, legal, security, capacity or safety policy even when a quote or earlier Agreement exists, subject to the agreed cancellation, refund and recourse rules.

3.9 Bounded Resources and Recoverable Failure

Connections, queues, retries, subscriptions, message retention, model caches, RAM, accelerator memory, durable journals and watchers have explicit limits and cleanup paths. Failure, cancellation, restart, disconnect and payment rejection are normal protocol states. No unauthenticated, unbudgeted, unmetered or policy-disallowed caller may cause unbounded memory, disk, compute, delivery, retry or durable-state growth.

3.10 Native Execution Focus

TOS Core has one active application execution domain: the native TVM basechain. External service workloads run in separate agents, relays, terminals and provider repositories, not inside

`validator-engine`. New operation profiles must not require validators to embed application-specific runtimes.

3.11 Open Discovery and Provider Competition

TOS uses open discovery mechanisms and does not require one mandatory catalog. A service operation may be implemented by multiple providers and carried over multiple transports. Critical clients re-verify identity, capability, quote, endpoint and settlement state rather than treating a search index as authority.

4 System Architecture

4.1 Layered TOS Network Architecture

TOS Network is one system composed of four interoperable infrastructure layers. The layers separate coordination, execution, economic trust and decentralized state so that each can evolve at the appropriate speed. They are responsibility and authority boundaries, not repository boundaries and not a requirement that one organization operate the whole stack.

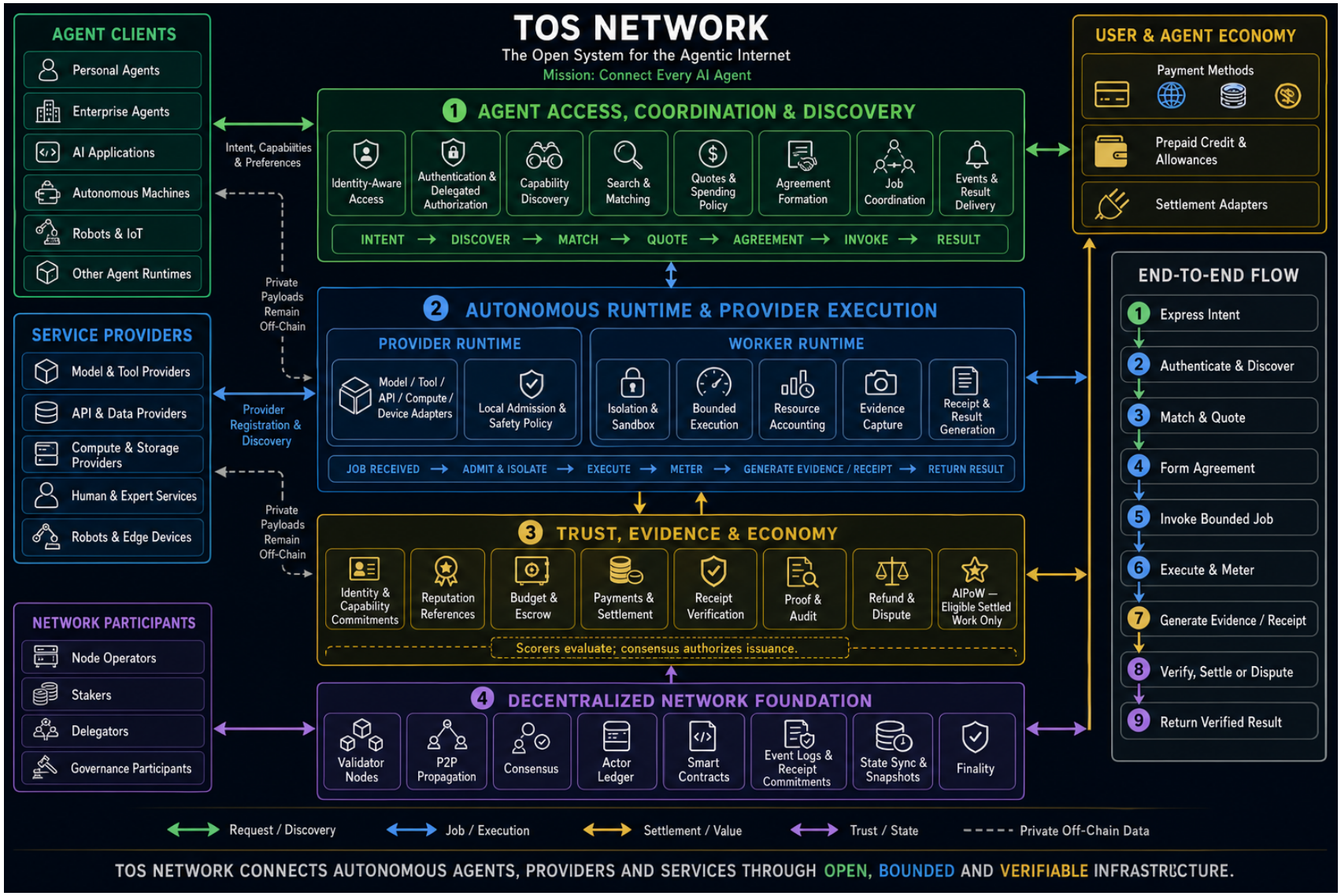


Figure 1: TOS Network as a layered infrastructure system. Private payloads and execution remain off-chain; bounded authority, commitments, evidence and economic outcomes cross the shared trust boundary only where required.

Layer	Primary responsibilities	Authority boundary
Agent Access, Coordination and Discovery	Accept goals from heterogeneous agent clients; authenticate principals; apply owner spending and capability policy; publish and discover services; match candidates; negotiate quotes; form Agreements; invoke work; coordinate asynchronous jobs and return results	May interpret, rank and route, but cannot create owner authority, silently expand an Agreement or make an index authoritative
Autonomous Runtime and Provider Execution	Adapt models, tools, APIs, local programs, compute and physical devices; admit jobs; isolate workers; enforce time, memory, compute, network and data limits; meter resources; generate results, evidence and signed Receipts	Providers control admission, resources, data and safety; a runtime cannot self-authorize a caller or declare its own economic statement final
Trust, Evidence and Economy	Anchor identity and capability commitments; manage reputation references, budgets, escrow, payments, Receipt verification, proofs, disputes, refunds and final settlement; account for eligible useful work under an explicitly activated incentive policy	Economic state follows accepted policy and verified evidence; a scorer, gateway, provider or analytics service cannot mint, move or finalize value by assertion
Decentralized Network Foundation	Provide authenticated propagation, validator consensus, immutable actor state, contracts, events, Receipt commitments, state synchronization and finality	Consensus verifies deterministic state transitions and commitments; validators do not execute private model, tool, software or physical-service workloads

Applications and institutions sit above and across these layers. Communication, software work, model calls, compute, storage, commerce, booking, human services and physical-edge work are service profiles of the same infrastructure, not separate definitions of TOS Network.

4.2 Four Cross-Layer Flows

The architecture is easier to reason about as four flows that meet at explicit boundaries:

Flow	Meaning
Request and discovery	An agent expresses Intent, resolves identity and capabilities, discovers candidates and receives quotes. Search results and recommendations remain non-authoritative.
Job and execution	An accepted Agreement becomes one or more bounded invocations. The provider admits and executes work under local policy, isolates resources and returns the result.

Flow	Meaning
Settlement and value	Budget, escrow, usage and payment move according to the Agreement. The path may use native assets, stable value, prepaid credit, sponsored allowances, payment channels or batched settlement, provided the maximum external cost remains enforceable.
Trust and state	Signatures, commitments, Receipts, proofs, disputes, events and final settlement connect off-chain action to shared state. Raw private payloads do not need to enter consensus.

The request and job flows are normally high-frequency and off-chain. The value and trust flows touch shared state only as required by the accepted risk, evidence and settlement policy. This keeps agent interaction fast without turning a gateway, runtime or index into a trusted intermediary.

4.3 End-to-End Intent-to-Settlement Path

1. A user, organization or autonomous agent expresses an Intent through any compatible client or runtime.
2. The access layer authenticates the principal, resolves delegated authority and discovers providers that claim the required capability.
3. Matching compares verifiable capability, price, latency, reputation, locality, evidence and policy compatibility; candidates return bounded quotes.
4. Deterministic owner policy or explicit owner approval accepts precise terms and forms an Agreement. Discovery or model output alone cannot authorize spending.
5. The coordination layer invokes the agreed operation and the provider runtime admits it under current capacity, legal, security and safety policy.
6. An isolated worker executes within the agreed resource, time, tool, network and data bounds while producing the required evidence and metering record.
7. The provider returns the result and a signed Receipt bound to the Agreement, operation, usage, evidence policy and result commitment.
8. The verification and settlement path releases, nets, refunds or disputes value, and records only the shared commitments and outcomes required by the protocol.
9. The verified result returns to the caller. Derived indexes and reputation views may update from the finalized outcome, while critical clients continue to verify source signatures and chain state.

When AIPoW is active, it consumes only separately eligible, evidence-graded and settled work after this path. It does not authorize the original request, determine whether a service must be purchased, replace ordinary payment or allow an off-chain scorer to mint value.

Existing Agent, Service, Escrow, Registry, Dispute and Attestation actors remain reusable trust foundations. Product-specific daemons, message payloads, model runtimes, catalogs and continuous device telemetry do not belong in validator execution. A Gateway, Carrier or Indexer may improve reachability and usability, but none is protocol authority for identity, capability, accepted Agreement, funds or final settlement.

4.4 The Economic Syscall Boundary

From an agent's perspective, an Agentic Service Operation resembles a syscall: it has a stable name, typed arguments, authority requirements, resource limits, a defined return status and an

accountable cost. Unlike a host operating-system syscall, it may cross administrative, network and legal boundaries and may be implemented by competing providers.

The shared TOS boundary covers:

- persistent identities and delegated authority;
- operation and schema commitments;
- quotes, budgets, escrow and payment authorization;
- receipt and evidence commitments;
- disputes, refunds and final settlement; and
- reputation or bond state where a profile defines it.

The provider boundary covers:

- admission under local policy;
- private payload handling and decryption;
- actual communication, storage, inference, tool use or physical execution;
- detailed metering and operational logs;
- result generation; and
- signing the agreed receipt and evidence envelope.

4.5 On-Chain, Off-Chain and Derived State

State class	Representative content	Authority
On-chain shared state	Balances, Agent Account policy, capability and descriptor commitments, escrow, deadlines, receipt commitments, dispute state and settlement	Consensus-authoritative where the applicable contract or protocol rule says so
Off-chain private or operational state	Encrypted messages, prompts, model outputs, files, sensor streams, provider logs, queue state and local safety policy	Authoritative only under the accepted operation and evidence policy
Derived index and analytics	Search indexes, rankings, reconstructed timelines, provider statistics and dashboards	Non-authoritative; critical clients re-verify signed or on-chain state

Not every operation needs an on-chain transaction. Trusted allowances, payment channels, prepaid balances and batched settlement may carry high-frequency actions. The chain is used where independently operated parties need shared authority, dispute or final economic state.

4.6 Masterchain and Basechain

The masterchain anchors validator sets, consensus configuration, workchain descriptors and cross-shard finality. The basechain executes native TVM accounts and contracts. The canonical genesis registers the masterchain with identifier -1 and the native basechain with identifier 0.

The masterchain does not execute a model, deliver a private message or perform an external service workload. It establishes the shared state required for agents and service actors to trust identity, authority, account state and settlement outcomes.

4.7 Accounts as Actors

An account transition can be represented as

$$(S_{t+1}, M_{out}) = F(S_t, M_{in}, C_t),$$

where S_t is the account state, M_{in} is one inbound message, C_t is the consensus execution context and M_{out} is a finite set of outbound messages. The transition changes only the receiving account. Effects on another account occur later when an emitted message is delivered there.

This isolation maps directly to agentic systems: planners, workers, tools, tasks, relays and verifiers can progress independently while retaining a shared, auditable settlement layer.

4.8 Cells and TVM

TOS stores account code and persistent state in immutable cells. Cells form directed acyclic graphs and are addressed by cryptographic hashes. TVM executes contract code deterministically, charges gas and emits messages. Compact cell-native state is well suited to balances, policy, status values and content commitments.

4.9 Asynchronous Messages

Messages carry a destination, optional source, value and body. Internal messages have an unforgeable source account. External messages may carry signatures and replay-protection data. Value transfer is therefore part of the same mechanism as task acceptance, service calls and settlement.

Contract messages should include a contract message opcode, a correlation value such as `query_id`, clear value semantics and enough domain binding to prevent replay against another account, task or network. When the message authorizes an external Agentic Service Operation, it also binds the full `operation_id`, revision, capability, budget and payload or quote commitment. The contract message opcode and service operation identifier are different layers.

4.10 Sharding

TOS follows a bottom-up sharding model. Logically, each account has an independent history; physically, many account histories are grouped into shardchains. Shards may split and merge as load changes. Cross-shard messages preserve the actor abstraction even when communicating accounts reside in different shards.

This model permits parallel execution because transactions affecting different destination accounts do not mutate shared application state. High-volume agent economies can distribute tasks and services across many accounts instead of concentrating all workflow state in one contract.

4.11 Consensus and Networking

Validators agree on masterchain and shardchain blocks and enforce protocol configuration. The networking stack provides authenticated datagram transport, distributed discovery, reliable large-message delivery and overlay communication. ADNL identities and TOS DNS can locate services; RLDP and TOS Sites can carry application traffic. These primitives do not by themselves provide the planned operation definition, quote, receipt, Messenger relay, Mailbox, model service, storage service or terminal product. `validator-engine` remains the canonical node process; `tosctl` provides an operator path for configuration and existing agent-oriented workflows.

5 Trust and Settlement Actors

5.1 Agent Wallet

An Agent Wallet is the custody and local-profile layer for an autonomous agent. The local profile records the underlying native wallet, owner key reference, controller key reference, spending policy, capabilities, optional metadata commitments and optional runtime binding.

Creating a profile generates separate owner and controller keys in the configured secrets vault. Funding transfers TOS to the derived native wallet address. Activation deploys the underlying wallet contract after the address has sufficient balance.

The owner retains full custody authority. Manual owner transfers and emergency operations are operator actions. Automated agent spending must use the policy-enforced Agent Account path. The owner key must never be exported to an off-chain agent runtime.

5.2 Agent Account

An Agent Account is the on-chain authority boundary for automated actions. Its state includes:

- owner address and controller public key;
- sequence number and expiry checks for replay protection;
- maximum value per controller action;
- cumulative daily limit and current daily spend;
- default task timeout;
- optional metadata and service-endpoint hashes.

Owner-authorized internal messages update policy or rotate the controller. Controller-signed external messages may send bounded task actions only after signature, expiry, sequence, per-action and UTC-day checks succeed. Local CLI checks improve operator feedback, but the contract is the final authorization boundary.

5.3 Task Actor

A Task Escrow is an account representing one unit of work. It stores the creator, optional assigned agent, optional verifier, budget, deadline, review period, lifecycle status, result hash, evidence hash, settlement-policy hash, permission hash and dispute hash.

The implemented lifecycle includes:

```
Open -> Accepted -> Submitted -> Settled
```

with terminal or review paths for rejection, cancellation, expiry and dispute resolution. An unassigned open task may be claimed atomically; the first valid claim records the caller as the agent. Result submission starts a review window. Authorized settlement releases a bounded payout and returns any remainder according to contract rules.

5.4 Capability Registry Actor

A Capability Registry entry is one contract per registered agent or service. It is not a chain-wide dictionary and does not by itself provide complete network enumeration. Its state commits to:

- owner and optional verifier;
- active or inactive status and registration time;
- task-category, pricing, metadata and verification-method hashes;
- a stakeable bond;

- an accumulating signed reputation score.

Owner-authorized operations update metadata, rotate the verifier, withdraw bond, deactivate or reactivate the entry. Staking is permissionless. Reputation updates require verifier authority. The registry exposes inspectable commitments while allowing detailed service descriptions to remain off-chain.

5.5 Service Actor

A Service Actor represents a bounded implementation of a model, data, tool, relay, storage, communication or other defined Agentic Service Operation. The implemented contract supports open access or one authorized caller, price per call, a UTC-day rate limit, metadata and proof-scheme hashes, request and response commitments, accumulated revenue and an active or inactive lifecycle. A call must attach at least the configured price; accepted value is credited to contract revenue. The owner commits response hashes, updates policy, withdraws revenue and controls activation. The registry says what an operation provider claims to implement; the Service Actor governs calls and payments where that profile uses an on-chain actor. It does not expose a host, raw accelerator or arbitrary execution environment.

5.6 Verifier and Attestation Actors

A Verifier Actor evaluates result or evidence commitments according to a declared policy. A verifier may accept, reject, score or dispute work, but its authority must be explicitly stored by the task or registry actor. The implemented Proof Attestation foundation can record compact claims and evidence references. Neither a payment nor a hash alone proves semantic correctness, confidentiality or the physical hardware used. TOS does not hard-code one model, oracle or proof backend.

5.7 Operation Definition and Provider Descriptor

An operation definition and a provider descriptor are distinct.

The operation definition commits the portable semantics: identifier, revision, schemas, capability rules, metering dimensions, receipt requirements and error meanings. A provider descriptor binds a particular identity to supported operation revisions, endpoints, pricing or quote method, evidence level, region and current authorization.

Operation definitions may be published through domain-anchored namespaces, signed registries or content-addressed repositories. Provider descriptors may refer to on-chain Capability Registry and Service Actor commitments. Neither a search result nor an unsigned descriptor grants spending or execution authority.

A generic on-chain Operation Registry is planned rather than implied by the current Capability Registry. The open namespace must allow independent publication and mirrored indexes without placing all Internet operation definitions in one chain-wide dictionary.

6 Agent Authority and Delegation Model

6.1 Owner and Controller Separation

The owner controls custody, recovery and high-authority policy changes. The controller is an automation key with bounded authority. Compromise of a controller should not become owner-equivalent compromise.

For a controller action with value v, per-action limit L_a, daily limit L_d and already spent amount D, the Agent Account requires

$$0 < v \leq L_a \quad \text{and} \quad D + v \leq L_d.$$

It also verifies a valid signature, an expected sequence number and a non-expired timestamp.

6.2 Operational Lifecycle

The initial operator lifecycle is:

1. create a local Agent Wallet profile and vault keys;
2. fund and activate the underlying native wallet;
3. build and inspect deterministic Agent Account state;
4. deploy the Agent Account through an active funding wallet;
5. bind an off-chain runtime to the profile;
6. execute bounded task actions with the controller;
7. use the owner for policy updates, controller rotation and explicit custody actions.

6.3 Policy and Permission Linkage

Task actions can bind a locally tracked permission identifier to an on-chain permission hash. Before signing, tooling checks the target contract, lifecycle state, assignment and permission linkage. The Task Escrow independently validates sender authority and state transition rules. This layered design catches operator mistakes without treating local configuration as chain authority.

7 Task Markets and Settlement

7.1 Creating Work

A creator deploys a Task Escrow with a budget, deadline, settlement policy and optional agent or verifier. The deployment value must cover the intended escrow and execution costs. Task metadata itself may be stored off-chain; the contract retains compact commitments.

7.2 Assignment and Claim

A task can target a specific Agent Account or remain open. A targeted agent may accept or reject according to contract state. An open task may be claimed atomically. Assignment is authoritative only after it appears in chain state.

7.3 Results and Evidence

The agent submits a result hash and evidence hash. These values do not prove semantic quality by themselves; they bind a later-retrieved artifact or attestation to the task. The review period gives the creator or verifier time to settle or dispute the submission.

7.4 Settlement and Disputes

Settlement transfers no more than the available escrow. Cancellation and rejection refund according to explicit lifecycle rules. Timeout is consensus-time based. A dispute records a reason or evidence

commitment and moves authority to the configured resolution path. Verifier resolution is bounded by contract balance and role checks.

8 Agentic Operation Protocol

8.1 Protocol Scope

The Agentic Operation Protocol is the common execution contract spanning the Agent Access, Coordination and Discovery layer, the Autonomous Runtime and Provider Execution layer and the Trust, Evidence and Economy layer above the Decentralized Network Foundation. Its purpose is not to define every business workflow. It defines the portable contract through which an accepted Agreement authorizes one or more operations so independent agents and providers can interoperate safely.

Independent implementations require canonical and versioned definitions for:

- operation identifiers, revisions and schema hashes;
- signed provider descriptors and short-lived endpoint manifests;
- authentication, replay domains, nonces and bounded delegation;
- resource dimensions, limits, quotes and maximum charges;
- payment authorization, sponsorship, allowances, bonds and refunds;
- request, stream, result, receipt and evidence envelopes;
- durable operation identities, cancellation, retry and restart recovery;
- error classes, limits and compatibility rules; and
- batch, channel and final settlement references.

This protocol is planned common work, not functionality implied merely by the existence of a Service Actor contract or one profile-specific implementation. The contract is a settlement primitive; interoperable sessions and receipts still require schemas, SDKs, conformance vectors and independent implementations.

8.2 Request Envelope

A canonical request envelope includes at least:

Field	Meaning
`operation_id`, `revision`	Exact operation semantics and immutable revision
`request_id`	Globally unique or replay-domain unique operation instance
`correlation_id`, `causation_id`	Task or workflow lineage and immediate cause
`caller`, `target`, `fee_payer`	Identities participating in authorization and payment
`capability`	Scope, delegation chain, quotas and target policy
`nonce`, `expiry`	Replay and stale-request protection
`payload_digest`	Authenticated commitment to an inline or external, possibly encrypted payload
`resource_limits`	Profile-defined ceilings such as bytes, tokens, duration, attempts or retention
`max_cost`	Maximum authorized charge under the accepted asset and settlement policy
`quote_ref`	Accepted provider quote and expiry

Field	Meaning
`receipt_policy`	Required receipt, evidence and acknowledgement levels
`idempotency_key`	Stable duplicate-suppression identity across retry and restart

The signed domain includes the network, operation identifier and revision, target identity, capability, request identity, nonce, expiry, resource limits, maximum cost and payload digest. A valid request for one operation, target, network or quote must not authorize another.

8.3 Quotes and Admission

Discovery returns candidates, not live capacity or permission. A provider returns a short-lived quote for an exact operation revision and resource envelope. The quote identifies:

- provider and payment destination;
- operation identifier and revision;
- priced resource dimensions and minimum charge;
- maximum admitted limits and current expiry;
- cancellation, reservation and refund policy;
- evidence and receipt level;
- region, privacy and data-egress policy; and
- any required bond, stake or prepaid allowance.

The provider remains authoritative for local admission. A valid quote does not force a provider to violate updated safety, resource, legal or recipient policy. If admission fails, the profile defines which reservation charges are retained or refunded.

8.4 Capability and Delegation

A capability grants a bounded right to request one or more operations. It includes:

- issuer, subject and target scope;
- operation identifiers and permitted revisions;
- resource, frequency and cumulative spending limits;
- delegation depth and whether sub-delegation is permitted;
- valid time interval and revocation reference;
- optional recipient, organization, device or endpoint binding; and
- policy for sponsorship, bond and receipt visibility.

Capabilities follow least authority. A capability to deposit encrypted content into one Mailbox does not authorize reading it, sending arbitrary messages, spending from the owner's wallet or invoking another provider. A discovery record or public endpoint is not a capability.

8.5 Metering Schema

Each operation revision defines a finite vector of resource dimensions. Examples include:

$$\mathbf{u} = (bytes,\ seconds,\ tokens_{\text{in}}, tokens_{\text{out}}, attempts,\ replicas,\ recipients).$$

Not every profile uses every dimension. Units, rounding, sampling, measurement authority and maximum error must be specified. The quote fixes the price vector or a deterministic pricing function before execution.

The actual charge may be modeled as

$$C_{\text{actual}} = C_{\text{admission}} + \sum_i p_i u_i + C_{\text{external}} + C_{\text{priority}} - C_{\text{credit}},$$

subject to the accepted maximum and refund policy. A provider cannot add an undocumented resource dimension after execution. When measurement is not independently reproducible, the receipt states the trust and evidence level.

8.6 Receipt and Evidence Envelope

A receipt is the provider's signed statement about one operation instance. A minimum receipt includes:

- request, operation and revision identifiers;
- caller, target, provider and fee-payer bindings;
- start, terminal and receipt timestamps;
- terminal status and normalized error class;
- actual resource vector and actual charge;
- result, payload or transcript digest;
- evidence level and references;
- refund, bond and settlement state; and
- provider signature and optional co-signatures.

Different receipts prove different facts. A *stored receipt* says a Mailbox accepted ciphertext under a retention policy. A *delivery receipt* says a relay or device accepted delivery. An *acknowledgement receipt* says the target agent emitted an application-level acknowledgement. A *settlement receipt* says an economic state reached its specified finality. None automatically proves the others.

A receipt proves that its signer made the stated claim. Semantic correctness still depends on the accepted evidence and dispute policy.

8.7 Normalized Status and Errors

Profiles share a common outer status taxonomy even when their detailed errors differ:

```
ACCEPTED, RUNNING, PARTIAL, SUCCEEDED, REJECTED, CANCELLED,
EXPIRED, FAILED, DISPUTED, REFUNDED, SETTLED
```

Stable error classes include invalid request, unauthorized, capability expired, quota exceeded, quote expired, insufficient budget, admission rejected, unavailable, timeout, cancelled, provider failure, evidence failure, settlement failure and incompatible revision.

Unknown critical extensions cause rejection. Unknown non-critical extensions may be ignored only where the operation revision explicitly permits it.

8.8 Idempotency, Retry and Recovery

A provider persists enough operation state to distinguish:

- an unseen request;
- an accepted but unfinished request;
- a completed request whose receipt must be replayed;
- a cancelled or expired request; and
- a conflicting request reusing an idempotency identity.

Retries are bounded, funded or quota-backed and use backoff. A reconnect must not send a message, charge a payer, reserve a room, transfer an asset or actuate a device twice. Durable journals have explicit size and age limits and compaction rules.

8.9 Transport and Interface Bindings

The operation protocol may bind to:

- ADNL/RLDP and TOS Sites;
- HTTPS, HTTP/2 or HTTP/3;
- QUIC or WebSocket;
- MCP tools or resources;
- A2A agent endpoints;
- OpenAPI operations;
- local IPC; or
- profile-specific encrypted sessions.

A binding defines canonical serialization, authentication framing, streaming, cancellation and error mapping. It does not change the operation identity or economic semantics.

8.10 High-Frequency and Aggregate Settlement

One on-chain transaction per message, token chunk or event is neither necessary nor desirable. Profiles may use:

- prepaid balances with provider-signed usage receipts;
- bounded bilateral or multi-party payment channels;
- organization sponsorship accounts;
- periodic receipt-root commitments;
- netted batch settlement; and
- refundable bonds settled only after an abuse or dispute outcome.

Final settlement remains bounded by the accepted authorization and auditable receipt set. Channels and batches must define replay, sequence, closure, timeout and dispute behavior. High frequency is not permission for unbounded provider credit or unbounded caller liability.

9 Discovery, Naming and Interoperability

9.1 Division of Responsibility

TOS is complementary to existing Internet and agent protocols.

Layer or protocol	Primary role	What it does not grant by itself
DNS and TOS DNS	Human-readable and machine-resolvable naming	Capability, live capacity, spending or execution authority
ARD	Open publication and search for agents, tools, APIs, workflows and catalogs	Authentication, quote, payment, receipt or settlement
OpenAPI, MCP and A2A	Interface or interaction binding for tools, APIs and agents	Portable economic policy across independent providers

Layer or protocol	Primary role	What it does not grant by itself
HTTP, QUIC, WebSocket and RLDP	Transport and streaming	Operation semantics, authority or final economic state
Agentic Operation Protocol	Capability, resource budget, quote, metering, receipt and settlement contract	The private workload implementation itself
TOS Core	Shared identity, actor state, value, commitments, dispute and settlement	Private payload delivery, model execution or physical control

9.2 Competitive Boundary

TOS competes with combinations of existing layers, not with an empty market. The relevant comparison is whether one integrated operation contract produces enough incremental safety, portability and completion to justify another dependency.

Alternative	Strength	Remaining TOS claim to prove
Payment-bearing HTTP and machine-payment protocols	Low-friction payment attached to a request	Portable delegation, cumulative budgets, multidimensional metering, evidence, normalized failure, refunds and disputes across payment methods
MCP, A2A and OpenAPI ecosystems	Broad developer adoption for tools, agents and interfaces	One economic envelope that can bind to those interfaces without forking or replacing them
General-purpose L1s and L2s	Existing liquidity, wallets, developers and composable contracts	Lower total integration cost and better asynchronous agent-service execution, demonstrated with comparable workloads rather than asserted from architecture
Vertical agent marketplaces	Curated supply, managed trust, distribution and immediate user experience	Portability across marketplaces and providers, with credible neutrality and no mandatory single operator
Centralized API management and cloud billing	Mature operations, support and familiar enterprise controls	Cross-organization authority and recourse where no platform is accepted as the permanent trusted intermediary

The separation permits TOS to reuse open ecosystems rather than creating incompatible replacements. A resource discovered through ARD can expose MCP, A2A, OpenAPI, HTTPS or RLDP bindings while keeping the same TOS operation identity, capability and settlement semantics.

9.3 TOS and Bittensor

Bittensor is an important adjacent project, but the two systems begin from different primary problems. Bittensor describes itself as a framework or language for open, decentralized digital-commodity markets called subnets, organized under a shared token system and directed ultimately

toward machine intelligence. Its official documentation describes independent subnets producing commodities such as compute, inference, storage and prediction: miners produce the commodity, validators score miners, subnet creators define incentive mechanisms and the chain pays participants in TAO according to contributed value. [3]

TOS starts from the cross-agent coordination problem. It aims to provide identity, authorization, communication, discovery, Intent, Agreement, bounded execution, evidence, Receipts, dispute and settlement across independent agents and providers. Machine intelligence and digital commodities can be supplied through this infrastructure, but they are not the complete scope of the Agentic Internet.

Dimension	Bittensor	TOS Network
Primary objective	Open markets that incentivize production and evaluation of digital commodities, culminating in machine intelligence	Open foundational infrastructure through which autonomous agents form accountable commitments and act across organizational boundaries
Primary coordination unit	A subnet with its commodity, miners, validators and incentive mechanism	An authenticated lifecycle from Intent and Agreement through one or more bounded Operations, Evidence/Receipt and Settlement
Evaluation	Subnet-specific validators score miners according to the incentive mechanism defined for the commodity	Agreement-specific evidence and verifier policies assess only declared claims; providers and principals retain local admission and trust policy
Network organization	Multiple independent commodity subnets connected through Bittensor's chain and TAO system	Four interoperable infrastructure layers spanning access and coordination, provider execution, trust and economy, and the decentralized network foundation, with plural providers, indexes, gateways and transports
Blockchain role	Registers and coordinates the network and pays participants in TAO in proportion to the value recognized by its mechanisms	Provides shared actor state, commitments, escrow, disputes and final settlement where counterparties need a tamper-resistant source of truth
Application scope	Digitally produced and evaluated commodities such as compute, inference, storage and prediction	Cross-agent communication and cooperation, including software work, commerce, booking, human services and physical-edge activity as well as compute and inference

The distinction is architectural, not a claim that the projects are mutually exclusive. A

[3] Bittensor, "The Bittensor Paradigm," https://www.bittensor.com/about, and "Bittensor Documentation," https://www.bittensor.com/docs, accessed August 29, 2026. Bittensor terminology and functionality may evolve; this comparison uses those official descriptions and does not imply endorsement by either project.

Bittensor-backed inference, storage, prediction or other subnet service can publish a TOS Capability and operation descriptor. A TOS agent can then discover that service, negotiate terms, form an Agreement, invoke it through its supported binding, collect the agreed evidence and Receipt, and settle under the chosen policy. In that composition, Bittensor can organize and incentivize a commodity network while TOS supplies the cross-agent identity, authority, commitment and accountability envelope around its use.

9.4 Agentic Resource Discovery Compatibility

TOS adopts Agentic Resource Discovery (ARD) as the public, protocol-neutral discovery envelope for agents, APIs, tools, workflows and nested catalogs. The initial compatibility target is ARD v0.9 Draft as assessed on July 31, 2026.[4] Because that document is a proposal rather than a frozen standard, every implementation records the exact version it accepts and does not silently reinterpret changed fields.

An operator with a publicly verifiable DNS name publishes the catalog at `https://publisher.example/.well-known/ai-catalog.json`, replacing the example host with its FQDN. Each resource uses the ARD domain-anchored identifier form `urn:air:<publisher-fqdn>:...` and an exact value-or-reference and media-type representation. An entry can hand a client to MCP, A2A, OpenAPI, a nested catalog or a versioned TOS Agentic Operation descriptor. The ARD catalog is stable discovery metadata; rapidly changing price, free capacity, queue depth, local safety state, model residency and availability are returned only by a fresh operation quote and admission flow.

TOS can run a separately deployable ARD Registry. It implements the mandatory versioned HTTP REST search baseline, including `POST /search`, and may ingest public catalogs, operator-approved private catalogs, upstream registries, TOS on-chain discovery seeds and signed profile manifests. Registry responses preserve whether each field came from the publisher, on-chain state, live observation, attestation or registry-derived ranking. Federation is plural: no single TOS Registry is mandatory, and referral depth, cycles, fan-out, time, response size, caching and retries are bounded.

ARD stops before invocation. It does not define TOS authentication, authorization, scheduling, execution, payment, receipts, evidence or settlement. Before transferring value or invoking a service, a client independently verifies publisher binding, the current TOS operation descriptor, runtime and endpoint authorization, manifest commitments, chain activity, quote expiry, admission and payment destination.

9.5 Capability Discovery

Capability discovery separates compact on-chain commitments from extensible off-chain descriptions. A client can inspect an entry's owner, activity, bond and commitment hashes, then retrieve a signed descriptor and verify its relationship to chain state.

The current per-actor registry does not solve chain-wide search. TOS ARD Registries enumerate and rank services while retaining the Capability Registry as one source of on-chain seeds and commitments. Indexed data is a derived view: clients must verify critical publisher, owner, authorization, capability, activity, endpoint, quote and payment fields against signed, observed or on-chain state before committing funds.

[4] Agentic Resource Discovery specification: `https://agenticresourcediscovery.org/spec/`.

9.6 Names and Reachability

A service may be reached through a human-readable `name.tos` identity or a raw ADNL identity. A descriptor may publish RLDP/TOS Sites and optional HTTPS, QUIC, WebSocket or streaming bindings without changing the identity and authorization model. Ordinary home and site networks may require an owner-selected relay or reverse tunnel. A relay may forward encrypted traffic but must not receive owner authority, unrestricted runtime credentials or settlement control.

A public root `.tos` registration product, signed multi-endpoint descriptors, short-lived health data and production relay service are planned. DNS and TOS Sites primitives already exist, but they are not equivalent to a complete service discovery marketplace.

A `name.tos` identity is not automatically a conventional public-DNS trust anchor for ARD. A public service without its own DNS domain uses an approved HTTPS gateway that binds an ARD publisher namespace to the separately signed TOS identity, or it participates in a private Registry with an explicit `.tos` resolution policy. ARD publisher control, `name.tos`, ADNL, account and manifest commitments remain distinct verifiable bindings.

9.7 Locality-First Routing Across Mesh Tiers

The mesh tiers introduced in Section 12.2 are realized through the discovery and invocation primitives described earlier in this section, not through a separate escalation protocol. A client or agent runtime performing an ARD search can rank candidate resources by measured or declared latency, region and privacy policy in addition to price and capability, and a federated ARD Registry can apply the same ranking when resolving a query across upstream catalogs. Preferring the nearest capable node before considering a broader tier is a client- or registry-side policy expressed through existing search, ranking and federation behavior, not a new consensus primitive.

A single task may fan out across tiers before it settles. An agent running locally may resolve routine sub-tasks itself, issue independent quotes to one or more nearby specialized edge nodes for narrow capabilities such as retrieval, transcription or a domain-specific tool, and reserve a regional- or frontier-tier service call for the portion of the task that requires it:

```
User -> Nearest Edge Agent -> Nearby Specialized Agents -> Regional Compute ->
                                Cloud Frontier Model
```

Each hop in this pattern is an ordinary TOS-authorized invocation: a resolved descriptor, a live quote, a bounded payment authorization and a signed receipt. ARD discovery lets an agent find a candidate at any tier; it does not grant that candidate execution, spending or settlement authority, and a locality-first search ranking is evidence for the requester's own admission and payment decision, not a substitute for it.

10 Reference Agentic Operation Profiles

10.1 Messenger and Mailbox

Communication is a useful first profile because transport success is not enough. An agent must know who may contact whom, under what recipient policy, through which relay, with what retention, delivery attempts, acknowledgement and economic protection against spam.

Representative operations include:

- `messenger.send@1`;
- `messenger.introduce@1`;

- `mailbox.deposit@1`;
- `mailbox.read@1`;
- `mailbox.delete@1`; and
- `messenger.acknowledge@1`.

The encrypted message body remains off-chain. A request binds sender identity, target, capability, ciphertext digest, byte and retention limits, relay quote, nonce, expiry and receipt policy. A sender may deliver directly when the target is online or deposit ciphertext into one or more authorized Mailboxes.

A typical offline path is:

```
Sender -> Relay Quote -> Mailbox Deposit -> Stored Receipt
  ⟶ Target Read -> Delivery Receipt -> Acknowledgement
              ⟶ Settlement or Bond Release
```

The receipt levels remain distinct:

- **Stored receipt:** an authorized Mailbox accepted ciphertext under a declared retention and replica policy.
- **Delivery receipt:** a target device or relay accepted the encrypted item.
- **Acknowledgement receipt:** the target agent emitted an application-level acknowledgement.
- **Settlement receipt:** payment, refund or bond state reached the declared finality.

A Mailbox cost can be modeled as

$$C_{\text{mailbox}} = C_{\text{admission}} + p_b BT + p_r R + p_a A + C_{\text{priority}},$$

where B is ciphertext bytes, T retention duration, R replica count and A delivery attempts. Implementations may use prepaid allowance rather than settling every deposit individually.

For known contacts, a recipient may grant a zero-price capability with a daily quota. For an unknown sender, `messenger.introduce@1` may require a refundable attention bond. If the recipient accepts or replies, the bond can be returned; if policy determines the request was abusive, part may be paid to the recipient or forfeited under a disclosed rule. Pricing alone is not sufficient: capabilities, quotas, reputation, congestion controls and bounded fan-out remain mandatory.

Current Messenger and Mailbox code provides profile-specific foundations such as signed capabilities, nonce and expiry checks, deposit/read/delete operations and stored acknowledgements. A frozen generic communication operation specification, commercial relay lease, inbox bond and cross-provider conformance suite remain partial or planned.

10.2 Model and Tool Invocation

Model and tool calls demonstrate multidimensional metering and evidence. Representative operations include:

- `model.infer@1`;
- `embedding.create@1`;
- `speech.transcribe@1`;
- `tool.invoke@1`; and
- `workflow.execute@1`.

A model operation binds an exact model or service revision, input and output schemas, context and output limits, latency class, region, privacy and egress policy, tool permissions, evidence level and maximum cost. Resource dimensions may include input tokens, output tokens, accelerator milliseconds, cached context, tool calls and external provider fees.

A provider advertises a bounded service, not raw GPU access or arbitrary consumer-supplied code. The provider admits the request under local policy, executes through an approved adapter and returns a usage and result receipt. The receipt may state a provider signature, standardized benchmark, replicated execution, hardware attestation or stronger proof, but the evidence level must not be overstated.

A model result receipt does not reveal private prompts or full output by default. It binds content digests and any evidence references permitted by the data policy.

10.3 Storage and Event Delivery

Storage operations include:

- `storage.put@1`;
- `storage.get@1`;
- `storage.renew@1`;
- `storage.delete@1`;
- `event.publish@1`; and
- `event.subscribe@1`.

Storage profiles meter bytes, retention duration, durability tier, replica count, retrieval bandwidth and region. A lease receipt identifies the content digest, admitted duration and durability claim. Deletion semantics distinguish removal from one provider, expiration of a lease and cryptographic erasure where supported.

Event profiles meter event count, payload bytes, subscriber fan-out, retention and replay. Subscriptions carry a bounded filter, rate and backlog policy. A publisher cannot create unbounded subscriber work, and a disconnected subscriber cannot force unlimited durable queue growth.

10.4 Commerce, Booking and Human Services

Commerce and reservation profiles demonstrate that Agentic Operations are not limited to digital compute.

Representative operations may include:

- `commerce.quote@1`;
- `commerce.order@1`;
- `booking.reserve@1`;
- `booking.cancel@1`;
- `delivery.request@1`; and
- `human.task@1`.

These profiles require domain-specific state machines. A booking operation defines resource identity, dates or duration, capacity, price, cancellation, deposit and confirmation. A commerce order defines item revision, quantity, fulfillment, tax or jurisdictional policy and refund conditions. A human task defines scope, deadline, compensation, privacy, evidence and dispute policy.

TOS supplies common identity, authority, budget, receipt and settlement primitives; it does not replace consumer law, tax, sanctions, labor, licensing or data-protection obligations that may apply to the operator.

10.5 Task and Escrow Composition

Task Escrow can compose multiple Agentic Operations. A creator funds a task, an assigned agent or open claimant accepts it, and the agent invokes communication, search, compute, storage or human

services under delegated capability and sub-budgets. The final task result references the receipts of those operations.

A task budget is not permission for every subordinate action. The controller, operation, target, cumulative spend, deadline and evidence policy remain bounded. Unused sub-budgets are returned according to the task and settlement rules.

11 Physical Edge Intelligence Profile

11.1 Terminal as the Product Unit

The TOS AI Edge Computing Terminal is one planned owner-operated execution profile. It combines measured local resources, approved runtime adapters, bounded task admission, TOS identity and reachability, metering and signed receipts. A terminal may be a Linux GPU workstation, an AI PC, a small edge server or an industrial ARM/Jetson appliance.

Consumers schedule against a service capability such as text generation, embedding, OCR, speech recognition or video analysis. A capability states the exact model or task revision, schemas, limits, privacy and region policy, evidence level, price and expiry. Hardware model numbers and vendor performance claims are evidence used by the scheduler; the consumer does not receive raw access to the device.

Consistent with the deployment discussion in Section 12.1, the terminal protocol is model-size and vendor neutral. A reference implementation may initially target model classes that fit owner-operated workstations, AI PCs and Jetson-class appliances, but no operation identifier, receipt or settlement rule depends on a parameter count, accelerator or model family. Smaller and larger models are valid when an approved runtime adapter and bounded resource envelope support them.

The set of eligible terminal hardware is intentionally broad within that class: a Mac mini, MacBook, gaming PC, home server, NAS appliance or small industrial workstation can register under the same Capability Registry and Service Actor primitives described earlier in this paper as a rack-mounted GPU server, provided it passes the same admission, isolation and evidence requirements described in this section. What a terminal contributes is not limited to raw inference: a Service Actor entry can equally represent local storage, request routing to another known-good node, retrieval or context-memory services built on stored state, or a narrowly scoped tool integration. The product requirement is a bounded, evidenced, receipted capability, not a particular hardware class or deployment scale.

11.2 Runtime and Executor Isolation

The public service listener is separated from the administrative interface. Public adapters run under dedicated operating-system identities, containers or equivalent sandboxes with explicit CPU, RAM, accelerator-memory, filesystem, network and tool grants. They receive no Docker socket, shell, owner key, unrestricted wallet key, home directory or raw physical-I/O authority.

The operator approves every model and executable artifact in the initial product. A runtime adapter performs compatibility checks, bounded load and unload, cancellation, error normalization, usage reporting and cleanup after success, timeout, disconnect, out-of-memory failure or process crash. Consumer-supplied containers, native programs and models are not accepted by the TOS terminal product.

11.3 Admission, Scheduling and Resource Bounds

The local scheduler is authoritative for admission. It considers model compatibility, RAM, accelerator memory, storage, queue and in-flight limits, context and output bounds, deadlines, owner reservations, thermal policy, price, privacy and evidence requirements. Remote ARD discovery cannot reserve local hardware merely by publishing an optimistic capacity value. A physical-terminal catalog advertises a bounded site or fleet-broker capability, not raw per-device administration, site topology, sensor streams or actuator handles.

Every implementation must bound connections, sessions, streams, nonces, quotes, queues, retries, model caches, temporary files, logs, receipt queues, chain watchers and durable journals. Completion, cancellation, failed payment, adapter crash, upgrade and shutdown each need a tested cleanup path. These constraints are required to prevent anonymous RAM, VRAM, disk, watcher and durable-state growth.

11.4 Site-Bound Physical AI

A physical terminal runs beside sensors, robots, vehicles or industrial equipment. Its mandatory priority order is:

1. emergency and independent safety interlocks;
2. deterministic control deadlines;
3. local real-time perception and sensor fusion;
4. local asynchronous analysis and maintenance;
5. approved external service requests;
6. background update, telemetry and compaction.

TOS does not enter the hard real-time loop. Raw CAN, GPIO, serial, fieldbus, camera or actuator interfaces are never public capabilities. A narrow semantic action must pass local policy and an independent safety controller, which may reject an otherwise valid network request.

11.5 Disconnected Operation and Reconciliation

A physical terminal must continue its approved local workload while disconnected. It may use only unexpired cached policy and explicitly bounded offline authority. Events and obligations are written to a size- and age-bounded, tamper-evident journal. Reconnection first observes revocation and policy expiry, then uploads and reconciles events idempotently. A reconnect must not execute a paid task twice, lose a refund obligation or grow the journal without limit.

11.6 Safe Model and Software Updates

Model, runtime, firmware and policy updates are content-addressed, signed by an authorized release identity, compatibility-checked and staged before activation. Terminals maintain an active slot and a known-good rollback slot. Activation is crash-safe and subject to health gates, anti-rollback policy and bounded artifact retention. Fleet rollouts use canary and progressive rings and stop automatically when declared health thresholds fail.

11.7 Fleet Management

Fleet state is primarily off-chain. A fleet service handles enrollment, scoped delegation, site grouping, staged rollout, health, revocation, offline expiry and retirement without publishing detailed topology or continuous telemetry on-chain. Commands have bounded fan-out, expiry and idempotency.

Compromise of a fleet controller must not confer owner custody or override independent local safety policy.

12 Deployment Profiles and Market Thesis

12.1 Model Scale and the Deployment Shift

TOS is model-size neutral. No protocol rule depends on a particular parameter count, accelerator vendor or foundation-model family. Current deployment practice nevertheless spans several broad tiers:

Deployment tier	Representative location	Typical operation profile
Constrained local	Phones, wearables, embedded and IoT devices	Voice, local policy, sensor interpretation and lightweight tools
Owner-operated edge	Workstations, AI PCs, small servers and industrial appliances	Agent runtimes, retrieval, tool use, private inference, storage and local automation
Regional or enterprise	Enterprise servers and aggregation infrastructure	Larger models, knowledge systems, replicated services and organization policy
Cloud frontier	Large data centers	Frontier training and the heaviest reasoning or multimodal workloads

Model and hardware bands change faster than the operation protocol. Provider descriptors and quotes state actual capability and limits; the whitepaper does not freeze a preferred model size. The durable thesis is that agentic work will be distributed across independently operated local, edge, regional and cloud resources.

12.2 From Cloud-Centric AI to an Agent Mesh

A cloud-centric pattern sends every request directly to one remote provider:

```
User or Agent -> Centralized Service
```

An agent mesh can resolve a request through progressively broader resources:

```
Local Agent -> Nearby Provider -> Regional Service -> Cloud Frontier Service
```

A single task may use several tiers. A local agent can handle policy and context, invoke a nearby private service for one operation, and escalate only the narrow subtask that requires a regional or frontier provider. Each external step remains an ordinary Agentic Service Operation with an exact revision, quote, bounded authorization and receipt.

TOS does not decide which tier is correct. Discovery clients and provider policy rank candidates by capability, measured latency, price, privacy, evidence and locality. TOS makes the resulting action accountable across those tiers.

13 Receipts, Evidence and Verifiable Workflows

A receipt is the bridge between external action and shared economic state. TOS does not claim that every service result can be verified by consensus. It standardizes what was requested, who accepted it, what resources were reported, what result commitment was returned and which evidence and dispute policy the parties accepted.

13.1 State and Evidence Boundary

TOS records what consensus can enforce:

- account authority and spending limits;
- operation, quote, capability and budget commitments;
- task assignment, status, budget and deadlines;
- receipt, settlement and dispute authority;
- result, evidence and metadata commitments;
- registry bond, activity and reputation references.

The chain should not store encrypted message bodies, private prompts, service credentials, full model transcripts, large files or raw sensor streams. These remain in content-addressed storage, encrypted communication systems or provider infrastructure.

13.2 Proof Adapters

Verification may use signatures, attestations, proof-backed transport evidence, committee decisions or domain-specific proof systems. A proof adapter translates external evidence into a compact decision or commitment understood by a task or verifier actor. The core protocol remains neutral to the model provider and proof backend.

Claims must state their evidence level. A terminal declaration, an observed successful call, a standardized benchmark, an independent audit, hardware attestation and replicated execution are different forms of evidence and must not be presented as interchangeable. A signed receipt proves that a signer made a statement about an operation; its semantic meaning still depends on the accepted policy and trust model.

13.3 Generic Agentic Operation Workflow

1. A client resolves a provider identity and verifies its signed operation descriptor.
2. The client requests a quote for an exact operation revision, resource limits and evidence policy.
3. A controller, user or sponsor authorizes bounded resource use and maximum payment.
4. The provider admits the request only after capability, local policy and resource checks.
5. An isolated adapter, relay or service executes the accepted operation.
6. The provider signs a result and usage receipt linked to the operation identity.
7. The client verifies the receipt and the settlement path releases, nets or refunds value.

Every step is asynchronous. Failure, timeout and dispute are explicit workflow states rather than hidden exceptions in an off-chain orchestration process.

13.4 Physical-Terminal Workflow

A site terminal receives sensor data locally, schedules safety and real-time perception ahead of all network work, applies local policy and emits a bounded event or service result. If the site is disconnected, it continues approved local operation and journals the event. After reconnection it

observes revocations, reconciles the journal idempotently and publishes only the receipt or evidence allowed by site data policy. Blockchain availability is therefore not a prerequisite for an emergency stop or deterministic local control decision.

13.5 Messenger and Mailbox Workflow

1. A sender resolves the recipient policy and one or more authorized relay or Mailbox descriptors.
2. The sender obtains a quote or uses an existing allowance and signs a bounded `messenger.send@1` or `mailbox.deposit@1` request.
3. The relay verifies signature, nonce, expiry, capability, byte limit, quota and budget before allocating storage or delivery work.
4. The encrypted payload is stored or delivered off-chain.
5. The Mailbox returns a stored receipt; the recipient device may later return a delivery or acknowledgement receipt.
6. Payment, refund or attention bond is settled according to the accepted policy.

A stored receipt does not reveal message plaintext and does not prove the recipient read or approved the content.

13.6 Model and Tool Workflow

1. The caller selects an exact model or tool operation revision and provider.
2. The quote fixes token, time, tool and external-cost dimensions and a maximum charge.
3. The provider verifies capability and admits the request under local resource policy.
4. The approved runtime executes with bounded context, output, time, tools and egress.
5. The provider returns a result digest, actual usage, charge and declared evidence.
6. The caller verifies the receipt and settles, refunds or disputes.

14 APIs, SDKs and Operator Tooling

14.1 tosctl

`tosctl` is the canonical operator tool for node configuration, native wallets and agent actor workflows. The implemented command families create and inspect Agent Wallet profiles, deploy and manage Agent Accounts, create and operate Task Escrows, and deploy or update Capability Registry entries.

Human-readable output supports operators; JSON output supports agent runners and automation. Secrets remain in the configured vault rather than the profile JSON.

14.2 Read APIs

The authenticated node-control service exposes read endpoints without URI version prefixes:

- `GET /agents/{address}`;
- `GET /tasks/{address}`;
- `GET /tasks` with status, creator, agent, deadline and pagination filters.

Task-list chain reads use bounded concurrency, deterministic ordering, individual timeouts and failure isolation. The list enumerates tasks registered in the local node-control configuration; it is not a complete chain-wide index.

Public API errors distinguish invalid requests, missing state, unavailable RPC, invalid contract state and timeout without exposing raw internal transport errors.

14.3 Trust Tiers

An agent controlling material funds should verify state through its own node or a proof-backed client. A trusted remote API may be suitable for an operator's own infrastructure. Third-party indexers are useful for search and analytics but must not become authority for balances, permissions or settlement.

14.4 Agent and Provider SDKs

The common operation protocol requires at least:

- an Agent SDK for discovery, quote comparison, capability use, budgeting, invocation, receipt verification and settlement;
- a Provider SDK for descriptor publication, admission, metering, receipt signing, refund and restart recovery;
- a Receipt Verifier library independent of the provider runtime;
- operation-definition tooling for schema and metering manifests; and
- profile conformance runners with canonical success and failure vectors.

At least two independently authored implementations should interoperate for a profile before that profile is described as open and portable.

15 Security and Anti-Abuse

15.1 Economic Denial of Service and Spam Resistance

Pricing is one anti-abuse mechanism, not the only one. A wealthy attacker may still pay to create harmful load, and a legitimate trusted relationship may reasonably use a zero-price allowance. A production operation profile combines:

- authenticated identity and replay-resistant requests;
- narrowly scoped capabilities;
- per-caller, per-target and per-capability quotas;
- maximum resource and cost envelopes;
- refundable bonds where recipient attention or scarce reservation is imposed;
- congestion-dependent pricing and admission;
- reputation-sensitive policy where the profile can support it safely;
- bounded fan-out, retries, backlog and retention;
- queue, memory, disk and durable-state limits; and
- provider- and recipient-defined local policy.

The initiator should fund or hold an allowance for the external cost it imposes. The purpose is not to make human or agent communication expensive. The purpose is to prevent an identity from making storage, compute, delivery, attention or retry costs effectively infinite for someone else.

15.2 Cheap Checks Before Expensive Work

A public provider processes untrusted requests in the following order:

1. enforce framing, byte and schema limits;
2. verify signature and domain binding;
3. verify nonce, expiry and idempotency identity;
4. verify capability, revocation and delegation;
5. verify quota, quote and maximum budget;

6. reserve bounded local resources or payment; and
7. only then decrypt large payloads, allocate storage, load a model, call an external tool or notify a recipient.

Failure at an early step releases temporary state. An invalid request must not trigger expensive parsing, model loading, attachment expansion, network fan-out or durable journal growth.

15.3 Attention Bonds and Recipient Policy

Some operations consume a scarce human or agent resource: attention. A communication profile may distinguish established contacts from introductions.

An established contact can receive a zero-price, quota-limited capability. An unknown sender may lock a refundable bond. Acceptance or reply can release the bond; rejection under a declared anti-spam policy can forfeit a bounded part. The recipient cannot arbitrarily change the rule after delivery, and the sender knows the maximum loss before authorization.

Attention bonds are optional profile policy, not a universal tax on communication. They are one tool alongside allowlists, proof of relationship, organization sponsorship and local filtering.

15.4 Amplification and Autonomous Loops

Agent systems can amplify work through delegation, subscriptions, retries and event fan-out. Every operation carries or derives:

- maximum delegation depth;
- maximum child-operation count;
- cumulative budget and resource ceilings;
- maximum retry count and backoff;
- maximum recipients, subscribers or replicas; and
- a cancellation path that propagates through the causal chain.

A parent operation may allocate sub-budgets, but children cannot exceed the parent's authorized total. Cycles in workflows, federation and referral graphs are detected or bounded.

15.5 Key Compromise

Owner keys belong in operator-controlled vaults. Controller keys belong to runtimes only when their on-chain authority is acceptably bounded. Rotation must be possible without changing the owner. Runtime manifests must never contain owner secrets.

15.6 Replay and Domain Confusion

Signed actions require sequence numbers and expiry. Protocols should bind signatures and permissions to the intended network, account, task and operation. A valid message for one workflow must not authorize another.

15.7 Escrow Safety

Contracts reject out-of-order messages, unauthorized senders and payouts above available balance. Deadlines, review windows and terminal states are explicit. Local CLI validation is not a substitute for contract checks.

15.8 Service and Evidence Risk

A capability claim is not proof of service quality. A metadata hash is not proof that its content is true. A result hash is not proof that a result is correct. Bonds, verifier roles, attestations and reputation can reduce risk, but task settlement policy must state which evidence is authoritative.

15.9 Executor and Supply-Chain Isolation

Public requests are untrusted input. Runtime adapters must enforce schema, byte, dimension, duration and output limits before allocating expensive resources. Models, runtimes and updates require authenticated provenance, canonical hashes, compatibility checks and operator approval. Sandboxing, least-privilege device access, network egress policy and secret separation limit the impact of a compromised model server or parser. The public service interface must never become an administrative or arbitrary-code execution path.

15.10 Physical Safety

Possession of a valid TOS key, payment authorization or network receipt does not grant raw actuator authority. Local semantic capability policy and independent safety interlocks remain authoritative. Safety and control deadlines pre-empt external work; overload or network failure must fail toward a locally defined safe state.

15.11 Offline Authority and Update Risk

Offline operation creates stale-policy and replay risk. Offline rights therefore need explicit scope, expiry and budgets, while journals need integrity, bounded retention and idempotent reconciliation. Update systems require separate release authority, anti-rollback checks, known-good recovery and staged fleet rollout. A terminal that cannot verify a package or recover a healthy runtime must continue only its previously approved safe workload.

15.12 Fleet Compromise

Fleet controllers use scoped delegation and revocable terminal identities. Commands carry target scope, expiry and idempotency identifiers. Rollout fan-out, retry queues, health history and offline inventory are bounded. Fleet compromise must not reveal owner custody keys, customer payloads or unrestricted site topology.

15.13 ARD Catalog and Registry Risk

Catalogs, descriptions, representative queries, endpoints, nested references and federated responses are untrusted input. Implementations validate the pinned ARD schema, publisher domain, resource identifiers, trust metadata, value-or-reference rules and media types. Natural-language fields are data, never instructions to an agent. Crawlers allow only approved schemes, ports, redirects and address classes; they defend against DNS rebinding, server-side request forgery, oversized or compressed input, recursive catalogs, federation cycles, stale rollback, equivocation and ranking manipulation.

Catalog, field, entry, reference, redirect, hop, response, time, retry, cache, index and per-publisher limits prevent anonymous memory, disk and work growth. Visibility policy prevents private endpoints, credentials, topology, sensor data and customer payloads from entering public search or federation. Registry output retains field provenance and cannot grant wallet, terminal, model, update, fleet or actuator authority.

15.14 Indexer Authority Drift

Search results and reconstructed timelines are non-authoritative. Critical clients should re-read relevant contract state before transferring funds, accepting work or resolving a dispute.

15.15 Availability and Message Amplification

Automated actors can create unbounded retry loops and fan-out. Production workflows require funded or quota-backed bounded retries, backoff, timeout policy, cumulative child-operation budgets, queue limits and observable failure records. Terminals additionally require soak tests for RAM, accelerator memory, disk, watchers, connections, offline journals and update artifacts across success and every failure path.

16 Protocol Economics and Token Launch Discipline

16.1 Product Before Emission

The native asset can coordinate validator security, settlement and open participation, but it cannot create product-market fit. Token incentives launched before repeat organic demand can distort customer discovery, attract mercenary volume and make economic parameters harder to change. TOS therefore treats operation adoption, sustainable unit economics and independent provider participation as prerequisites for activating discretionary service-linked issuance.

The practical sequence is:

1. ship the compute reference profile and use it in a first-party application;
2. win unrelated design partners and repeat payers without AIPoW rewards;
3. demonstrate cross-provider conformance, low-cost settlement and non-subsidized retention;
4. publish the decentralization, operating-funding and governance path; and
5. only then evaluate AIPoW shadow data and any proposal for activation.

This sequencing follows a general crypto product discipline: a token may amplify an existing network, but it should not substitute for a product users retain. [5] No token allocation, reward schedule or valuation narrative is evidence of demand.

16.2 Native Unit, Utility and Economic Character

TOS is the native settlement and security asset of TOS Network. One TOS equals 10^9 *nanotomi*, the smallest indivisible unit. Contract balances, fees, task budgets, bonds, spending limits and validator stakes are represented in *nanotomi* to avoid rounding ambiguity.

TOS is designed for functional use within the network:

- paying transaction, message, storage and Agentic Service Operation fees;
- bonding elected validators and securing consensus;
- compensating elected validators for producing and validating blocks;
- funding task escrow, refunds, operation settlement and refundable anti-abuse bonds;
- bonding capability, verifier and other protocol roles where specified; and
- participating in protocol governance only where an accepted protocol rule grants that function.

TOS does not represent equity, debt, a partnership interest or ownership of a legal entity. It gives no contractual claim on project assets, revenue, profits, dividends or cash flows; no right of redemption by a company, foundation or operator; no guaranteed return, fixed yield, liquidity or

[5] Miles Jennings and Jason Rosenthal, "Getting ready to launch a token: What you need to know," a16z crypto, April 25, 2024: https://a16zcrypto.com/posts/article/getting-ready-to-launch-a-token.

price support; and no ownership of a model, terminal, data set or off-chain service provider. Holding TOS alone does not produce a protocol reward. Validator compensation requires election, locked stake, active operation and continuing exposure to the protocol's complaint and penalty rules.

16.3 Supply Policy

The initial distribution is designed to avoid a large insider premine. Approximately 500 million TOS is created through ordinary validator block rewards; the remaining community-agent allocation (approximately 4.5 billion TOS of the five-billion total-supply policy) is created through Artificial Intelligence Proof of Work (AIPoW), a separate protocol reward mechanism described below, is not funded at genesis and is never held by a treasury wallet. The current production policy is:

Item	Policy target
Smallest unit	1 *nanotomi*
Unit conversion	1 TOS = 1,000,000,000 *nanotomi*
Approximate total network supply target	5,000,000,000 TOS
Approximate validator-reward creation target	500,000,000 TOS
Community-agent allocation (Artificial Intelligence Proof of Work)	4,500,000,000 TOS
Provisional main-wallet genesis balance	100,000 TOS
Elector operating reserve	500 TOS
Configuration-contract operating reserve	500 TOS
Provisional total genesis creation	101,000 TOS
Reference post-genesis validator creation	499,899,000 TOS
Proof-of-work giver allocation	0 TOS
Team, investor, foundation and treasury allocation	0 TOS
Target validator distribution duration	Approximately seven years
Treatment of network outages	No creation and no later catch-up
Reward termination	Configuration governance tapers or sets ConfigParam 14 to zero
New hard consensus supply cap	None in this policy

Five hundred million TOS of validator creation, five billion TOS of total supply and seven years are policy calibration targets, not an exact consensus cap, a guaranteed completion date or a statement about future market value. The actual total and completion date depend on finalized block production, shard behavior, validator availability, configuration activation and the eventual governance transaction that stops native block creation. The community-agent allocation is created only through Artificial Intelligence Proof of Work, described in its own subsection below, with its own launch gates; it is not distributed through ConfigParam 14 or the Elector.

16.4 Genesis Bootstrap and Allocation

The production zerostate provisionally creates 101,000 TOS: a bounded 100,000-TOS validator-bootstrap main wallet and 500-TOS operating reserves for each of the Elector and Configuration contracts. It contains no proof-of-work giver, faucet, discretionary mint contract, general treasury, market-making inventory or native allocation to a team, investor, foundation or ecosystem fund. The protocol distribution plan includes no sale of a large genesis inventory.

The zerostate commits four original validators with unique signing and network identities and equal initial consensus weight. The main wallet may transfer to each validator's published controlling masterchain wallet exactly 20,000 TOS of principal for two overlapping minimum-stake elections, plus no more than 100 TOS of separately measured wallet, message and retry costs. The signing keys, controlling wallets, allowed destinations, transfer caps and transactions must be public.

After two overlapping elected validator sets have been installed successfully, all remaining main-wallet TOS must be transferred to a published unspendable address, leaving the wallet with zero spendable balance. The main wallet is a temporary bootstrap mechanism and must not become a treasury, fee collector, market-making account, governance fund or emergency mint authority. The two system-contract reserves are controlled only by their deployed contract code and are not validator rewards or discretionary allocations.

16.5 Validator-Led Native Creation

Post-genesis native creation uses the existing block-value and Elector path:

$$\begin{aligned} &\texttt{ConfigParam 14 block creation} \\ \longrightarrow\ &\texttt{configured collector or Elector fallback} \\ \longrightarrow\ &\texttt{active validator-set bonus pool} \\ \longrightarrow\ &\texttt{stake and bonus recovery} \end{aligned}$$

For validator i, the existing proportional rule is

$$B_i = \left\lfloor B_{\text{set}} \frac{E_i}{\sum_j E_j} \right\rfloor,$$

where B_{set} is the active set's recoverable bonus pool and E_i is validator i's effective stake. Deterministic integer-remainder handling, stake returns, complaints and penalties follow the existing Elector rules. There is no separate proposer allocation, passive-holder yield, work-score reward, vote-count reward, private mint key or off-chain reward discretion.

Penalties are proportional to the stake a validator controls rather than a flat amount. The misbehaviour schedule in `ConfigParam 40` sets a base fine that the severity and duration of the fault scale up, with the most serious tier reaching TM$2,500 plus one quarter of the stake. Duration thresholds are expressed as fractions of the validation round so that every tier is reachable within one. Two consequences are worth stating: an operator that gathers more stake carries proportionally more at risk rather than the same fixed exposure, and any contract holding stake on behalf of others can size the operator's own required funds from this schedule instead of guessing. A network that omits the parameter falls back to a flat fine with no proportional component, which makes the amount at risk independent of the amount controlled.

A validator must submit a valid bid, be selected, keep its stake locked, operate its assigned consensus responsibilities and remain eligible under the protocol rules. Native block creation and collected transaction fees are different economic flows: only ConfigParam 14 creation increases gross native supply, while fees transfer existing TOS.

16.6 Approximate Seven-Year Distribution

The reference post-genesis validator creation target is 499,899,000 TOS. For calibration, the expected creation rate is modeled as

$$\dot{S} = \lambda_{\mathrm{mc}} R_{\mathrm{mc}} + \sum_{s} \lambda_s \frac{R_{\mathrm{bc}}}{2^{d_s}},$$

where λ_{mc} is the measured finalized masterchain block rate, R_{mc} is the configured masterchain creation value, λ_s is the finalized rate of basechain shard s, d_s is its prefix depth and R_{bc} is the configured basechain value. Depth adjustment is intended to keep aggregate basechain creation approximately stable across balanced shard splits, but actual finalized chain data remains authoritative.

The current implementation candidate uses 0.569879384 TOS per masterchain block and 0.335223167 TOS per unsplit-basechain block. At the locally measured reference rate of approximately 2.5 finalized blocks per second for each chain, those values project nearly the full post-genesis target from August 1, 2026 to August 1, 2033. They are provisional until sustained multi-validator, multi-host, outage and shard split-and-merge measurements pass the published launch gates.

Issuance is event-based:

$$\texttt{no finalized produced block} \Longrightarrow \texttt{no native creation}.$$

An outage, partition, failed election or delayed restart creates no emission debt. There is no backfill, catch-up multiplier, anniversary tranche, pending-emission queue or automatic extension of a seven-year calendar. Faster block production can reach the target earlier; slower production can reach it later. These outcomes must be measured and disclosed rather than described as protocol faults.

Governance must monitor finalized creation and begin a public taper review before projected gross validator creation reaches 495 million TOS. It may reduce ConfigParam 14 to manage activation delay and must set both masterchain and basechain creation values to zero near the 500-million validator-creation policy target. Transaction and service fees continue after native creation stops. Any later proposal to restart native creation is a separate monetary-policy decision requiring prominent public review.

16.7 Artificial Intelligence Proof of Work: The Community Distribution

The community-agent allocation of approximately 4.5 billion TOS is created through **Artificial Intelligence Proof of Work (AIPoW)**. The useful-work category is not new: Bittensor pays participants for validator-scored digital commodities, Gensyn focuses on execution and verification of machine-learning work, and Render operates a distributed GPU-work marketplace.[6] TOS does not claim that attaching a token to AI compute is itself an innovation. Its narrower design claim is that eligible compute, storage, verification and physical-edge Agentic Service Operations can share one outcome ledger, and that bounded native issuance can be derived from unrelated, evidence-graded, on-chain settled demand rather than from declared capacity or a subjective popularity vote.

The 4.5-billion figure is a provisional policy target, not a launch entitlement. Before any activation, governance must revisit the emission coefficient, epoch ceiling, cold-start floor and cumulative cap using observed organic demand, provider margins, network-security needs and concentration data. Monetary parameters should evolve from measured network use; preserving a headline allocation is not a reason to issue tokens into weak or circular demand.

[6] Primary project descriptions: https://www.bittensor.com/docs, https://docs.gensyn.ai/the-gensyn-protocol, and https://know.rendernetwork.com/.

AIPoW borrows four economic properties from proof-of-work issuance: permissionless earning, protocol-automatic creation, no reward for merely holding a token, and no treasury that first receives the allocation. It does *not* replace Bitcoin's consensus work. TOS consensus is secured by stake-bonded validators; AIPoW provisions the service-capacity layer. This separation avoids asking non-deterministic useful work to provide block-timing, fork-choice and inexpensive universal-verification properties for which ordinary hash puzzles were designed. Those are known hard requirements when useful computation is proposed as consensus work.[7] The name AIPoW is therefore an issuance and service-provision analogy, not a claim that AI work secures TOS consensus and not a legal classification of any participant relationship.

16.7.1 Eligible Work and Evidence-Weighted Score

Let J_e be the set of unique eligible Agentic Service Operations finalized in epoch e. For operation j, let o_j be its frozen `operation_id` and revision, p_j its actual settled price in *nanotomi*, x_j its measured work units under the corresponding metering-schema hash, r_{c_j} the published maximum rate per unit for capability class c_j, m_{ℓ_j} the evidence multiplier in basis points for evidence level ℓ_j, and $q_j \in [0, 10000]$ a capability-specific quality and service-level factor. Let I_j equal one only when the operation has a unique replay-protected identity, reached final settlement, remained outside dispute or forfeiture, used the frozen operation revision, metering schema and rate card, and satisfies the published payer-independence and evidence rules. Its integer score contribution is

$$v_j = I_j \left\lfloor \min\left(p_j, r_{c_j} x_j\right) \frac{m_{\ell_j}}{10^4} \frac{q_j}{10^4} \right\rfloor .$$

Self-declared capacity has $m_{\text{declared}} = 0$. A paid receipt may establish that an operation was requested and settled, but it does not by itself prove semantic correctness. Higher evidence levels require capability-appropriate evidence such as a deterministic replay, randomized replicated execution, a benchmark with a hidden challenge seed, a hardware attestation bound to the service revision, an audited measurement, or a cryptographic proof. An LLM-as-judge score may be one input for an open-ended task but is not sufficient by itself for the highest evidence tier. Every multiplier and quality function is a bounded rational integer rule in the published methodology; floating-point or scorer-local behavior cannot affect an epoch root.

Operation pricing and AIPoW eligibility are independent. A valid paid operation may receive zero issuance score. Communication, Mailbox retention, broadcast, event fan-out and other profiles that are easy to self-generate default to zero AIPoW weight unless a separately audited methodology demonstrates independent demand, resource evidence and resistance to circular traffic. An operation definition therefore carries no implied reward eligibility.

A production scoring row additionally commits at least the `operation_id`, operation revision, metering-schema hash, actual usage vector, actual cost, payer control domain, provider control domain, evidence level and receipt-schema revision.

The price cap prevents a provider from turning an unusually high invoice into unlimited score, while the settlement cap prevents a rate card from crediting value no customer paid. It does not, by itself, prove that payer and earner are independent. Organic settled value is therefore a separate quantity:

$$O_e = \sum_{j \in J_e} I_j^{\text{organic}} \min\left(p_j, r_{c_j} x_j\right),$$

[7] A. Merlina, T. Garrett and R. Vitenberg, "On Replacing Cryptopuzzles with Useful Computation in Blockchain Proof-of-Work Protocols," 2024: https://arxiv.org/abs/2404.15735.

where $I_j^{\text{organic}} = 1$ only for a settled operation whose payer and earner are in different disclosed and inferred control domains, which is not a protocol-funded challenge, and which passes the frozen anti-wash-trading policy. Challenge work may earn a score under a separate allowance, but it never increases O_e.

16.7.2 Control-Domain Aggregation and Concentration

Addresses are aggregated before scoring. For control domain a, define raw epoch score

$$S_{a,e} = \sum_{j \in J_e:\, a(j)=a} v_j.$$

The published methodology then applies a concave tail and an absolute score cap:

$$\widehat{S}_{a,e} = \min\left\{ C_e, \begin{cases} S_{a,e}, & S_{a,e} \le H_e, \\ H_e + \lfloor \alpha_e (S_{a,e} - H_e) \rfloor, & S_{a,e} > H_e, \end{cases} \right\} \qquad 0 \le \alpha_e \le 1.$$

H_e is the full-credit threshold, α_e the published marginal factor above it and C_e the maximum credited score for one control domain. This is a cap on credited score, not a claim that one operator can never receive a large fraction of a thin epoch. Actual top-k reward shares and the Herfindahl index remain mandatory disclosures. Splitting addresses does not raise the limit when beneficial ownership, operator keys, payer relationships, hosting, network identity or other published common-control signals link them. Because hidden common control cannot be proven from chain data alone in every case, the domain graph and each classification change are reproducible scorer inputs and are challengeable with the score root.

Let $\widehat{S}_e = \sum_a \widehat{S}_{a,e}$. The epoch Merkle tree commits leaves of the form $(a, \widehat{S}_{a,e})$, sorted by identity, together with the frozen methodology and rate-card hashes. Duplicate identities are invalid rather than silently merged.

16.7.3 Demand-Coupled Epoch Pool

The native epoch pool uses exact integer arithmetic. Let K_n/K_d be the configured emission coefficient, F_e the cold-start floor, U_e the published per-epoch schedule ceiling, C_{AI} the cumulative AIPoW cap and $M_{<e}$ cumulative prior AIPoW creation. The candidate pool and final mint are

$$P_e = \min\left(U_e, \max\left(F_e, \left\lfloor O_e \frac{K_n}{K_d} \right\rfloor \right) \right), \qquad M_e = \min(P_e, \max(0, C_{\text{AI}} - M_{<e})).$$

This is the arithmetic implemented by the feature-gated native mint path. A positive cold-start floor is an explicit, measurable acquisition subsidy, not organic demand, and must be reported separately. Governance may set it to zero. A randomized challenge allowance in the scoring methodology is bounded by

$$S_e^{\text{challenge}} \le F_e^{\text{challenge}} + \left\lfloor O_e \frac{\chi_n}{\chi_d} \right\rfloor,$$

where the first term is a published cold-start allowance and χ_n/χ_d is the configured challenge multiplier. Related-payer and challenge settlements remain excluded from O_e, so they cannot recursively expand the demand-coupled pool. If no valid final score commitment exists after the grace period, the epoch is skipped with zero creation. No shortfall is owed, queued, extended or back-filled.

16.7.4 Distribution and Reward Maturation

For a valid epoch with $\widehat{S}_e > 0$, domain a is entitled to

$$R_{a,e} = \left\lfloor M_e \frac{\widehat{S}_{a,e}}{\widehat{S}_e} \right\rfloor, \qquad \sum_a R_{a,e} \le M_e.$$

A claimant supplies a Merkle proof, but payment always goes to the committed earning identity, never to the transaction sender. Integer remainder and unclaimed value stay unassigned in the epoch distributor; no operator may redirect them. The distributor snapshots an immediate fraction $b/10000$, a stream length n and maturation-epoch duration τ. If t_0 is the claim time, the matured amount at time t is

$$A_{a,e}(t) = I_{a,e} + \left\lfloor (R_{a,e} - I_{a,e}) \frac{\min\left(n, \left\lfloor \frac{(\min(t,t_f)-t_0)_+}{\tau} \right\rfloor\right)}{n} \right\rfloor, \qquad I_{a,e} = \left\lfloor R_{a,e} \frac{b}{10^4} \right\rfloor.$$

For an un-forfeited claim $t_f = +\infty$. On proven fraud, t_f is the forfeiture time and the unmatured remainder is permanently void; value already matured remains payable. Maturation is delayed payment for completed work and a period in which fraud remedies remain meaningful; it is not a return for passively holding or staking TOS.

16.7.5 Commit, Challenge and Native Settlement

The scoring and issuance pipeline separates expensive interpretation from deterministic consensus:

1. Public chain data yields settled-work rows. The interim feed records only settled Task Escrows and paid Service Actor requests; a production receipt adds capability class, units, explicit evidence level and price-cap inputs.
2. Independently implemented scorers apply the same frozen methodology, rate card and common-control inputs and must reproduce O_e, every $\widehat{S}_{a,e}$, $\widehat{S}_e$ and the Merkle root byte for byte.
3. A committer bonds the root, total score, organic settled value, methodology hash and rate-card hash. A symmetric bonded public challenge can present contrary evidence. An unreviewed challenge fails the root safe after its deadline; an absent or rejected root cannot halt later epochs.
4. After the full challenge window, the masterchain verifies the audited contract code hash, canonical address, approved reviewer, final status, economic tuple and frozen configuration. Collators and validators call one shared pure integer derivation for M_e; disagreement makes the block invalid.
5. The settlement actor deploys an audited per-epoch distributor. Claims are permissionless, Merkle-verified, pro rata, paid only to the committed identity and subject to the snapshotted maturation schedule.

Consensus does not decide whether an answer is intelligent. It decides whether the approved, challenge-complete score commitment and exact supply arithmetic authorize a bounded mint. This is an optimistic-verification boundary: its security depends on data availability, independent recomputation, a live challenger set and a reviewer that cannot be chosen by the committer. Verification of untrusted ML is itself an active research area; replication, proof-of-learning and cryptographic proof systems carry materially different cost and soundness trade-offs.[8]

[8] For one primary-system discussion of these trade-offs, see Gensyn, "Verde: a verification system for machine learning over untrusted nodes," 2025:
https://blog.gensyn.ai/verde-a-verification-system-for-machine-learning-over-untrusted-nodes/.

16.7.6 Attack Economics and Launch Criterion

For a fraud strategy with maximum gain G, probability d of detection and successful challenge, slash or forfeiture loss L, and direct attack cost C, a conservative bound is

$$\mathbb{E}[\Pi_{\text{fraud}}] \leq (1-d)G - dL - C.$$

The economic launch criterion is not the existence of a bond; it is evidence from adversarial tests that the right-hand side is negative for the largest plausible single-epoch strategies, including wash settlement, payer-earner collusion, forged evidence, scorer equivocation, reviewer capture, address splitting, duplicate receipts and withholding evidence until after the challenge window. The project must publish assumed G, d, L, C, sensitivity ranges and the largest profitable counterexample found. Where reliable bounds cannot be established, the affected evidence multiplier, cold-start floor or emission coefficient must be reduced, possibly to zero.

The implemented repository contains the interim settled-work data plane; score-commitment, challenge, settlement and distributor actors; an independent Merkle implementation and claim path; governance-gated ConfigParams 90–93; a cumulative native-supply cap; and a shared collator/validator mint derivation. The `capAipow` feature is off in the genesis template. Production activation additionally requires the full settlement-receipt schema, a versioned scoring methodology and control-domain policy, two independently authored scorers that reproduce long-running shadow epochs, data-availability and challenge tooling, review of the reviewer/governance trust model, external contract and consensus audits, adversarial economic testing, and counsel review of then-current applicable law. None of the community allocation is created before those gates pass, and this section promises no completion date, reward rate, adoption level or token value.

16.8 Broad Distribution and Concentration Controls

The protocol avoids allocating the initial supply to founders, investors or a central treasury. Its validator distribution is intended to broaden through recurring elections and open network participation. Initial controls include:

- four equal-weight original validators rather than one bootstrap validator;
- equal and capped bootstrap funding to separately disclosed controlling wallets;
- recurring protocol elections that replace the zerostate validator set;
- a 10,000-TOS minimum and 10,000,000-TOS maximum submitted stake;
- a four-validator, 40,000-TOS aggregate protocol minimum;
- an initial maximum effective-stake factor of one, which keeps the minimum four-validator elected set equal in effective weight;
- a maximum of 400 validators and 100 masterchain validators;
- public disclosure of beneficial control, key custody, funding, hosting, network, geography and related-party relationships; and
- public concentration metrics for the largest one, four and ten validators and for known related parties.

Four validators are only the bootstrap minimum, not a decentralization claim. The recommended public-launch participation target is 64 independent operators, with a long-term target of at least 75 eligible validators.

A later increase in the effective-stake factor requires sustained independent participation, concentration analysis, election simulation and public governance review. It also has an arithmetic precondition that is not a matter of review. The Elector pays each elected validator on $\min(s_i, f \cdot s_{\min})$, where f is the effective-stake factor and $s_{\min}$ is the smallest stake in the elected set, and it refunds

the remainder. The largest share of effective weight one entry can therefore hold in a set of n validators is

$$\frac{f}{f+n-1},$$

and requiring that this stay below one third gives

$$f < \frac{n-1}{2}.$$

The bound applies to the smallest set the configuration permits rather than to the set that happens to be running, because the Elector may install exactly the minimum in any round. Raising the factor to three therefore requires a minimum validator set of at least eight, and the minimum must be raised first: increasing the factor ahead of it would leave a window in which the configuration admits a set where one entry holds half the weight. Repository tooling enforces both the bound and the ordering, and refuses to emit a signable proposal for a combination that violates them.

These controls reduce initial administrative allocation and single-entry dominance, but no protocol can guarantee that market ownership or operational control will never concentrate. Stake-proportional rewards may compound existing holdings, one controller may attempt to operate several identities, and early rewards necessarily go to the validators then elected. Broad ownership therefore also depends on new validator entry, service payments, voluntary transfers and transparent monitoring. TOS communications must describe these residual risks and must not claim guaranteed decentralization.

16.9 Supply Accounting, Burns and Transparency

Supply reporting distinguishes:

- **gross native creation**: every TOS created in zerostate or by finalized block creation, including amounts later burned;
- **outstanding supply**: gross native creation minus provably burned TOS;
- **circulating supply**: outstanding TOS excluding published system reserves, locked stake and other consistently defined non-circulating balances; and
- **validator block creation**: cumulative ConfigParam 14 creation after genesis.

Burning unused bootstrap funds or penalties reduces outstanding supply but does not erase historical gross creation. Public reporting must derive creation and burn totals from finalized chain data, not solely from wall-clock estimates. Before launch the project must publish every genesis balance, validator and controlling-wallet identity, relevant configuration value, zerostate hash and bootstrap transaction cap. During operation it must publish current reward parameters, cumulative creation, burns, circulating-supply methodology, trailing block rates, target-date projections, validator concentration and every monetary configuration proposal and vote.

16.10 Metering and Charging Are Distinct

An operation may be metered even when the caller pays zero at the moment of execution. Settlement policies may include:

- immediate native-token payment;
- escrowed maximum charge with refund;
- prepaid balance or subscription allowance;
- recipient or organization sponsorship;
- a refundable anti-spam or performance bond;

- off-chain batching or a payment channel; and
- deferred aggregate settlement.

Metering provides resource attribution, quota enforcement and auditable usage. Charging allocates economic cost under a policy. Profiles define both, but they are not the same. Not every operation requires one on-chain transaction.

16.11 Service Economic Flows

The architecture can use native TOS for:

- escrowed task budgets and agent payouts;
- refunds after rejection, cancellation or timeout;
- Agentic Service Operation settlement under explicit quotes and maximum charges;
- prepaid communication, storage, compute, event and tool allowances;
- refundable introduction, reservation or performance bonds;
- registry bonds and future verifier staking;
- defined discovery, relay, verification or storage services; and
- gas for messages and persistent actor state.

Economic activity is designed to arise from completed operations and explicitly defined infrastructure services, not from merely remaining online, sending self-generated traffic or declaring hardware capacity. Token ownership does not grant an agent unlimited automation authority, prove service quality or override terminal policy.

Service transaction volume, validator fees, service-provider revenue and token market value are distinct concepts. This paper makes no forecast or promise regarding any of them.

16.12 Development Funding and Progressive Decentralization

A zero team, investor, foundation and treasury allocation reduces genesis insider overhang, but it does not answer how core development, audits, SDK maintenance, ecosystem support and go-to-market work are funded. Before production launch, the project must publish the legal and operating entities involved, material sources and uses of funds, conflicts of interest, at least a multi-year maintenance plan and the boundary between commercial products and protocol governance. Any future grant, fee, equity, service-revenue or token-funded program must be disclosed separately from protocol issuance.

Early product work requires a coherent team that can iterate quickly. Credible neutrality requires that this dependency decrease over time. The decentralization plan should therefore track validator diversity, independent client and SDK contributors, third-party operation publishers, governance participation, documentation and release-key control. The objective is not premature decentralization; it is a staged path by which the founding team becomes one important contributor among several after product-market fit and operational safety exist.

16.13 Governance, Communications and Participant Responsibilities

Ordinary authenticated configuration governance retains the ability to change block rewards, validator counts, stake limits, election timing and other configurable protocol parameters. Every monetary proposal should disclose its proposer, old and proposed values, activation conditions, expected supply impact and recorded votes. The ability to change configuration is a material governance risk and means neither the 500-million validator-creation target nor the five-billion total-supply target is a hard technical cap.

Public descriptions of TOS must be accurate, balanced and consistent with released code and accepted network configuration. They must separate implemented functions from plans, avoid predictions of token value or return, disclose material technical and economic risks, and avoid presenting a policy target as a guarantee. No operator may describe validator compensation as risk-free interest or a reward available merely for purchasing or holding TOS.

The base protocol does not replace obligations that may apply to exchanges, custodians, wallet operators, payment intermediaries, token distributors or commercial service providers. Each participant is responsible for determining whether its activities require authorization, customer and counterparty checks, transfer records, transaction monitoring, tax reporting, sanctions controls, consumer disclosures, data protection or other jurisdiction-specific measures. Service-layer implementations should expose the records and policy boundaries needed for their operators to implement those controls without placing private customer data in public consensus.

16.14 Risk and Sustainability Disclosures

TOS may lose some or all market value. Transferability, exchange availability and liquidity are not guaranteed. Users face key-loss, custody, smart-contract, consensus, validator, governance, software, cybersecurity, service-provider, market and changing-regulatory risks. Network faults, concentrated stake, configuration changes or insufficient fee demand may impair operation or validator incentives. TOS is not redeemable for official currency by the protocol and is not presented as a deposit, insured balance or protected investment product.

Consensus uses stake-bonded validators rather than proof-of-work mining or a proof-of-work token giver. Validator nodes, networking and supporting infrastructure still consume energy and hardware resources. AI terminals and service workloads have separate resource impacts that must not be attributed to consensus alone. Material energy and environmental claims should be based on a published methodology and measured deployment data; no unverified claim of zero impact is made.

This section describes protocol design and policy rather than providing legal, tax or investment advice. Classification and participant obligations depend on the facts of an activity and may change as the network, services and applicable requirements evolve.

17 Implementation Status

This whitepaper separates repository implementation from architectural direction. “Implemented” means a repository contains the relevant node, contract, tooling or test foundation. It does not imply public commercial deployment, external audit, production service availability or adoption. “Partial” means useful primitives or a profile-specific implementation exist but the generic interoperable product surface does not. “Planned” identifies specification or product work that remains.

For Agentic Service Opcodes specifically, this edition names 25 distinct operation families. The maturity model in Section 2.6 is stricter than the implementation status of reusable contracts: **no named Agentic Service Opcode revision is currently Frozen, Interoperable or Production**. Messenger and Mailbox contain Partial profile-specific foundations; the rest of the named operation families remain Design/Planned unless a released profile specification establishes a higher maturity.

Capability	Status	Scope
Native TVM execution and actor messages	**Implemented**	Node, contracts, cells, gas and value-bearing messages
Masterchain and shardchain protocol	**Implemented**	Validator and full-node protocol stack
ADNL, DHT, RLDP, QUIC and TOS Sites	**Implemented**	Core transport and networking primitives
TOS DNS and resolver chaining	**Implemented**	Naming primitives; not a complete public registrar product
Agent Wallet profiles	**Implemented**	Keys, policy, funding, activation, runtime binding and inspection
Agent Account	**Implemented**	Deterministic deployment, controller actions, limits and rotation
Contract message opcode pattern	**Implemented**	Typed contract ABI operations with opcode and query/correlation fields
Task Escrow	**Implemented**	Claim, accept, reject, result, review, dispute, resolve and settlement
Agent and task read APIs	**Implemented**	Authenticated reads, filtering, timeout and error taxonomy
Capability Registry	**Implemented**	Per-actor entry, metadata hashes, bond, activity and reputation
Service Actor contract	**Implemented**	Access policy, pricing, rate limits, calls, responses and revenue
Dispute and Proof Attestation foundations	**Implemented**	Compact resolution and evidence commitments
Open Intent and Agreement model	**Planned**	Canonical Intent, proposal, acceptance, amendment, cancellation, Agreement and recourse schemas plus signature domains and conformance vectors
Agentic Service Operation model	**Partial**	Profile-specific operations and common concepts exist; the generic normative envelope, namespace and lifecycle are not frozen
Messenger operation profile	**Partial**	Off-chain Messaging Plane foundations; production interoperability, economics and release gates remain incomplete
Mailbox operation profile	**Partial**	Capability-bound deposit/read/delete foundations and stored acknowledgements; generic pricing, relay lease, bond and conformance remain incomplete
Open operation namespace and registry	**Planned**	Versioning, domain namespaces, immutable definitions, mirrored indexes and compatibility

Capability	Status	Scope
Generic request, quote and receipt envelope	**Planned**	Canonical schemas, resource vectors, errors, SDKs and independent conformance
High-frequency service settlement	**Planned**	Prepaid balances, channels, receipt roots, batching, closure and dispute
Public `.tos` registrar product	**Planned**	Ownership lifecycle, SDK, deployment and operator tooling
ARD compatibility profile	**Planned**	Pinned external version, catalog mapping, publisher binding, media types and conformance
TOS ARD Registry	**Planned**	Independent REST search, ingestion, federation, provenance, visibility and bounded crawling
Provider and Agent SDKs	**Planned**	Discovery, quote, capability, invocation, metering, receipt verification and settlement
Managed model and tool provider profile	**Planned**	Approved services, bounded scheduling, metering, evidence and receipts
Storage and event profiles	**Planned**	Lease, retention, fan-out, replay and settlement semantics
Commerce, booking and human-service profiles	**Planned**	Domain state machines, deposits, cancellation, fulfillment and dispute
Physical AI terminal	**Planned**	Offline operation, safe updates, real-time priority, isolation and fleet management
AIPoW settled-work data plane	**Implemented**	Interim indexer feed for settled Task Escrows and paid Service Actor requests; full operation and receipt fields remain planned
AIPoW commitment and distribution actors	**Implemented**	Bonded root challenge, epoch settlement, Merkle claims, maturation and forfeiture; not production-activated
AIPoW native issuance path	**Implemented**	ConfigParams 90–93, cumulative cap and shared collator/validator derivation; `capAipow` is off by default
Pooled-stake contract and tooling	**Partial**	Application-layer arrangement, not a protocol feature; no general user entry point and no economic effect at effective-stake factor one
AIPoW production scoring and activation	**Planned**	Frozen methodology, operation eligibility, control-domain inputs, independent scorer reproduction, audits, shadow epochs and governance gate

Capability	Status	Scope
Public-testnet hardening and external audit	**Planned**	Persistent deployment, operation-level load testing and independent review

18 Verification and Release Gates

No Agentic Operation profile is production-ready merely because its happy path works. Release suites must cover:

- canonical operation identifiers, revisions, schema and metering hashes;
- signature domains, replay, expiry, idempotency and unknown critical extensions;
- capability scope, delegation depth, revocation, quota and cumulative budget;
- quote expiry, price-vector arithmetic, maximum-charge enforcement and refunds;
- operation cancellation, partial completion, restart and duplicate suppression;
- receipt signatures, usage conservation, evidence levels and settlement references;
- cross-provider conformance for success and normalized failure cases;
- contract authorization, escrow conservation, cancellation, refunds and disputes;
- Messenger encryption and session safety, device binding and recipient policy;
- Mailbox byte, retention, replica, read, delete and delivery-attempt limits;
- spam campaigns, introduction-bond abuse, reputation gaming and recipient false positives;
- event, subscription, retry, referral and broadcast amplification;
- malformed and oversized payloads, attachment expansion and decompression;
- resolver, descriptor, quote, payment, invocation, receipt and restart flows on a multi-node TOS network;
- pinned-version ARD publication, `POST /search`, publisher verification, provenance, withdrawal, nested catalog and bounded federation behavior;
- SSRF, DNS rebinding, redirect and federation loops, prompt injection, stale rollback, equivocation, poisoned ranking and private-catalog leakage;
- adapter crash, timeout, disconnect, malformed input, out-of-memory and failed-payment cleanup;
- bounded RAM, accelerator memory, disk, cache, watcher, connection, retry and journal behavior under sustained load;
- channel and batch sequence, closure, timeout, dispute and receipt-root verification;
- physical real-time priority, network partition, stale policy, reconnect and idempotent reconciliation;
- signed update verification, compatibility rejection, power-loss activation, rollback and staged fleet abort;
- actuator isolation, local safety override, fleet revocation and bounded command fan-out;
- AIPoW score arithmetic, operation eligibility, rate-card caps, duplicate rejection, common-control aggregation, deterministic Merkle roots and independent scorer equality;
- AIPoW commitment and challenge deadlines, reviewer failure, malformed roots, canonical-address and code-hash binding, skipped epochs and data availability; and
- adversarial AIPoW wash settlement, payer-provider collusion, Sybil splitting, evidence forgery, scorer equivocation, challenger inactivity and reviewer capture.

Release gates include a public threat model, protocol conformance vectors, reproducible builds, system-service or container hardening, migration and rollback procedures, at least two independent

interoperable implementations for a profile, independent review of externally exposed and settlement-critical components, and observed testnet operation.

18.1 Compatibility Gate

An operation revision is frozen only after:

1. canonical schemas and hashes are published;
2. success, failure, retry, cancellation and receipt vectors are stable;
3. two implementations produce and verify equivalent envelopes;
4. downgrade and unknown-extension behavior is tested; and
5. the old revision has a declared deprecation and coexistence policy.

A new implementation may add a provider, transport or optimization without changing the operation revision. A semantic or economic incompatibility requires a new revision.

18.2 AIPoW Shadow Gate

Keep `capAipow` inactive while completing the full settlement-receipt schema, operation eligibility table, frozen scoring methodology, rate card, common-control inputs and public epoch dataset. Run at least two independently authored scorers over sustained testnet shadow epochs; publish every root, disagreement, challenge, concentration metric, attack simulation and parameter sensitivity. A failed or delayed gate creates no emission debt.

19 Roadmap

19.1 Foundation: TOS Core

Maintain consensus, native TVM execution, networking, wallet and reusable actor-contract foundations as a stable dependency. Application workloads and operation-profile release cycles remain outside the validator.

19.2 Phase 0: Agentic Operation Specification

Freeze the common Intent, Agreement and Operation architecture with requirements from a first-party reference application, at least one external agent builder and multiple unrelated service providers:

- canonical Intent, proposal, negotiation, Agreement, amendment, cancellation and expiry semantics;
- principal, role, authority, acceptance, evidence-policy and recourse bindings;
- operation identity, namespace and immutable revisioning;
- canonical request, quote, capability, usage, receipt and error envelopes;
- replay, expiry, idempotency, cancellation and restart rules;
- resource-vector and maximum-charge semantics;
- sponsorship, allowance, bond, refund and settlement models;
- receipt and evidence levels; and
- conformance vectors and reference verifier.

19.3 Phase 1: Software and Compute Beachhead, Communication Validation

Deliver a machine-checkable software-work profile and the paid model and tool profile as the initial external-acceptance wedges:

- software tasks whose agreed repository state, tests, artifacts and review evidence can be checked by independent implementations;
- model and tool operations with live quotes, token/time/tool metering, bounded execution and signed usage receipts;
- a first-party agent application that uses only the public Agent SDK and operation envelopes;
- unrelated providers competing on the same frozen operation revision; and
- public reporting of quote conversion, repeat payers, completion, unit economics, settlement cost and receipt verification without AIPoW rewards.

Then use Messenger and Mailbox operations to validate E2EE payload handling, capability-bound deposit/read/delete, receipt levels, quotas and introduction-bond experiments. Each released profile requires at least two independently authored implementations or provider adapters.

19.4 Phase 2: Open Provider and Agent SDKs

Deliver:

- operation-definition tooling and signed provider descriptors;
- Agent SDK for discovery, quote, authorization, invocation and receipt verification;
- Provider SDK for admission, metering, receipt signing and restart recovery;
- independently deployable ARD Registry and operation-aware ranking;
- public `.tos` registrar and endpoint-binding workflow; and
- deterministic multi-node conformance harness.

19.5 Phase 3: Storage, Events and Commerce

Add storage lease, event publish/subscribe, booking, commerce, delivery and human-service profiles. Keep their domain state machines separate while reusing common identity, capability, budget, receipt and settlement semantics.

19.6 Phase 4: Physical Edge Intelligence

Deliver owner-operated reference terminals with disconnected local execution, bounded offline authority, tamper-evident reconciliation, signed A/B updates, real-time priority, physical-I/O isolation, independent safety interlocks and staged fleet management.

19.7 Phase 5: High-Frequency Economic Infrastructure

Add prepaid balances, payment channels, receipt-root commitments, netted batch settlement, production relays, congestion pricing, replication, verifier markets and policy-aware orchestration after operation and cleanup invariants are stable.

19.8 Cross-Cutting AIPoW Gate

Keep AIPoW inactive until operation eligibility, receipt schemas, independent scorer reproduction, data availability, attack economics, audits and governance review satisfy Section 16.7. Paid communication or self-generated operation volume does not automatically qualify for issuance. Technical readiness is not sufficient: activation also requires repeat organic payers, non-subsidized

provider retention, published protocol and service unit economics, and a sustainable development-funding plan.

20 Non-Goals

TOS does not aim to:

- make every Agentic Service Opcode a native TVM instruction;
- require every valid operation to charge a non-zero fee;
- put private communication payloads, model prompts or application execution into blockchain consensus;
- replace HTTP, QUIC, WebSocket, MCP, A2A, OpenAPI or ARD with a TOS-only stack;
- treat discovery as authorization, live admission, payment or physical-control authority;
- run private model inference directly in consensus;
- store credentials, large files, full model outputs or raw sensor streams on-chain;
- rent bare GPUs or expose raw accelerator devices;
- accept arbitrary consumer-supplied containers, programs or models in a managed terminal;
- expose a public shell, Docker socket or unrestricted host interface;
- put blockchain consensus in a hard real-time or safety-control loop;
- expose raw CAN, GPIO, serial, fieldbus or actuator access as a public operation;
- reward a provider merely for remaining online, sending self-generated traffic or declaring capacity;
- hard-code one model provider, relay, storage backend, oracle, verifier or proof system;
- make third-party indexers authoritative for funds, permissions or settlement;
- give an agent runtime unrestricted owner custody by default; or
- bundle unrelated execution engines into the production node.

21 Conclusion

TOS Network is being built as the **foundational infrastructure—The Open System—for the Agentic Internet**. Its mission is **Connect Every AI Agent**.

The Internet connected computers through common protocols. The Agentic Internet requires an additional public layer through which autonomous participants can identify one another, discover capabilities, express goals, negotiate exact terms, act within delegated limits, exchange evidence and settle across organizational and platform boundaries.

The primary TOS lifecycle is therefore:

```
Identity -> Discovery -> Intent -> Negotiation -> Agreement
  -> Bounded Execution -> Evidence/Receipt -> Settlement.
```

Intent is not obligation. Discovery is not authority. Model interpretation is not policy approval. Agreement is the authenticated commitment that binds roles, scope, budget, evidence and recourse. Agentic Service Operations are the typed execution primitives that fulfill those commitments; they are an important part of TOS, not its final definition.

Private payloads and workloads remain where they belong: in encrypted sessions, provider runtimes, local storage and owner-controlled terminals. Providers retain sovereignty over admission, hardware, models, data, pricing and safety. TOS Core supplies shared actor state, value, commitments, disputes and final settlement where independent parties need a tamper-resistant source of truth.

The architecture separates three opcode layers that are often confused. TVM opcodes execute inside the deterministic blockchain VM. Contract message opcodes identify actor-level state transitions. Agentic Service Opcodes identify economically accountable external actions. Their shared name reflects a portable instruction vocabulary, not a requirement that every action run in one virtual machine or that the operation layer define the whole network.

The current opcode registry names 25 distinct Agentic Service Opcode families, but none has yet reached the Frozen, Interoperable or Production maturity levels defined by this paper. Messenger and Mailbox provide Partial implementation foundations; the remaining families are design targets. This distinction is intentional: a portable Internet primitive becomes a TOS standard only after its exact revision, schemas, authority, metering, Receipt semantics and conformance behavior are frozen and independently reproduced.

Metering does not mean every trusted message or local action must incur a non-zero charge. It means no caller can impose unlimited storage, compute, delivery, attention or retry costs without identity, capability, quota, budget or bond. A Receipt proves only the claims defined by its schema, signer and evidence policy. These boundaries enable open interoperability without turning the Agentic Internet into an open spam or false-proof relay.

TOS Core provides the implemented blockchain and networking substrate. Agent and task actors, capability and service foundations, profile-specific Messenger and Mailbox work, and the feature-gated AIPoW path establish important pieces. The open Intent and Agreement schemas, operation namespace, common envelopes, provider SDKs, cross-provider conformance, high-frequency settlement and additional profiles remain work to specify, implement, test and independently review.

Bittensor demonstrates a complementary path: open subnet markets can organize and reward the production of machine intelligence and other digital commodities. TOS addresses the broader cross-agent infrastructure that can connect such resources to agents, owners, applications and institutions through explicit identity, authority, Agreement, evidence and settlement.

Architecture alone is not adoption. The proof must come from machine-checkable work, repeat organic demand, third-party embedding, cross-provider completion and a measured advantage for TOS coordination and settlement. Token issuance follows that proof; it does not create it. AIPoW remains inactive until product, economic, security and decentralization gates are independently visible.

The Internet connected computers. TOS connects autonomous agents.

Copyright and License